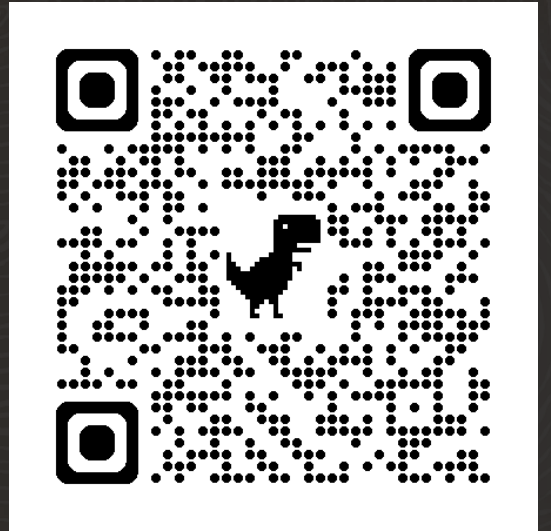




<https://github.com/albertattard/demo>

Examples



Java with AI Workshop

AI for Everyday Engineering

Albert Attard

Principal Java Success Consultant

Java Success Consultancy Team - Oracle

Aug 2026



Who am I?

Who am I?



- Albert Attard
- Born in Malta (a long time ago)
- Passionate Java Developer
- Moved to Köln, Germany, in Oct. 2018
- Joined Oracle in July 2022, as part of the Java Success Consultancy team

Who am I?

Where is Malta?



What do you expect to gain from this session about Java with AI?

Agenda



- Why this matters
- How it works
- Daily engineering practice
- Scaling responsibly

Outcome

- Give AI the context it needs
- Use it to explore, plan, build, and review
- Verify its output before relying on it
- Choose one low-risk workflow to try in your own team

AI is not a replacement for Software Engineering

AI changes how we solve software engineering problems.

Mastery takes about 10,000 hours...

Good news: We're not doing all that today.

You are not expected to become an expert after this workshop

but rather to simply identify gaps in your knowledge and begin a conversation

*“If there’s something strange, in your **Java**
code, who you gonna call?”*

Java Success Team!



Why this matters

A normal Java change involves more than writing code

- Add an optional delivery instruction to an existing order service

Find where orders are
created and validated

Understand existing
behaviour and
constraints

Decide what needs to
change

Implement the slice

Add or update tests

Review the diff and
make it safe to release

The hidden risks are often the questions we did not know to ask

- Could a downstream service reject, ignore, or mishandle it?
- Is the value subject to storage, audit, security, or retention rules?
- What assumptions are undocumented, or only known by someone else?

AI can shorten the uncertainty loop



Engineering question	AI can help you
Where does this behaviour live?	Navigate and summarise relevant code
What might break?	Identify callers, assumptions, and test gaps
What are plausible solutions?	Compare implementation approaches
Is this change credible?	Generate review questions and suggest verification



AI doesn't know what it cannot see



- AI can only reason about what it can see
- Its answers are limited by the context, documentation, and code it has actually inspected

AI changes the workflow, not the accountability



- AI can propose, explain, transform, and critique
- AI cannot know your unstated constraints or accept production risk
- A fluent answer is not evidence that a change is correct

Why this matters



- AI is valuable because much of an engineer's work is reducing uncertainty around a change, not merely typing code
- It can reduce that uncertainty, but only with relevant context and human verification

How it works

Typical LLM Behaviour (circa 2020)



Can two siblings each be taller
than the other?

Yes, two siblings can each be
taller than the other.

AI

Typical LLM Behaviour (circa 2020)



On a map, which compass direction is usually left?

On a map, north is usually to the left.

AI

Typical LLM Behaviour (circa 2020)



Which was invented first, cars,
ships, or planes?

Cars were invented first, followed
by planes and ships.

AI

ChatGPT Goes Public

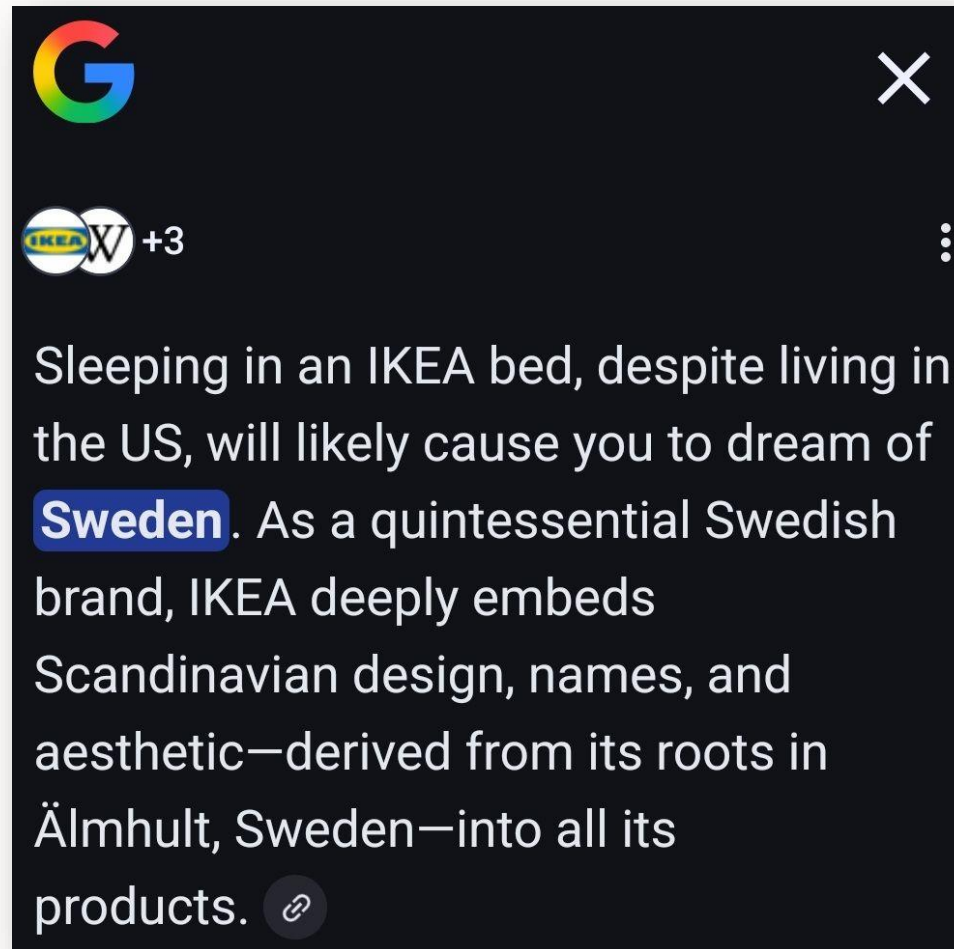
- ChatGPT was officially released by OpenAI on November 30, 2022, as a free public research preview running on an upgraded version of the GPT-3.5 architecture

Typical LLM Behaviour (circa 2024)

- By 2024, state-of-the-art LLMs could solve Olympiad-level maths problems involving techniques such as modular arithmetic, Wilson's theorem, and Zsigmondy's theorem



Typical LLM Behaviour (circa 2024)



Typical LLM Behaviour (circa 2024)

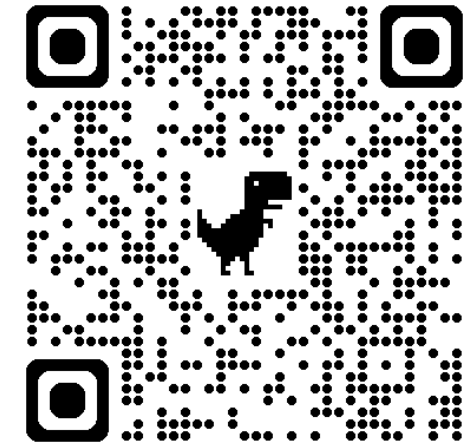
"What if Pythagoras had failed to install a security update? Be specific."



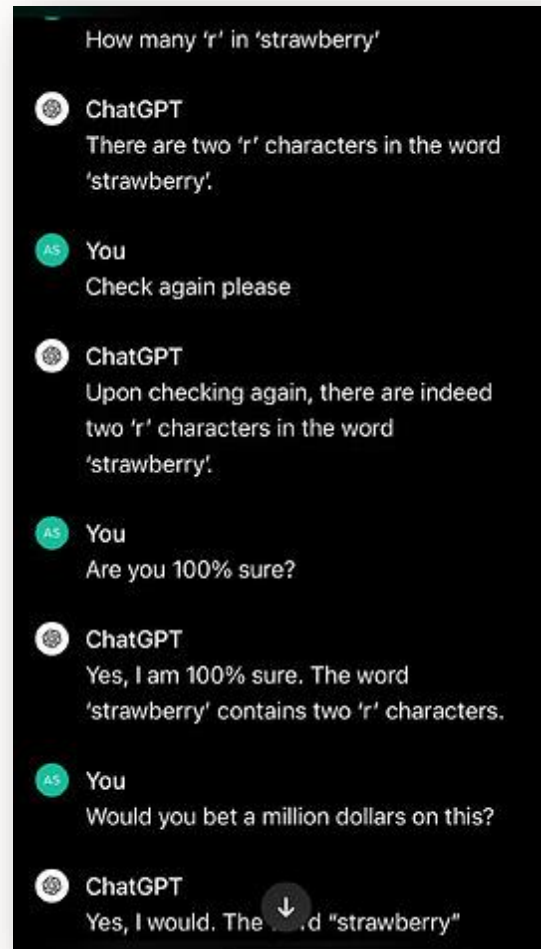
If Pythagoras had failed to install a critical security update for his surveying or modeling software (e.g., [Pythagoras CAD+GIS software](#)), he would likely have faced unauthorized access, data corruption, or system crashes, leading to inaccurate calculations for his geometric theorems and disruption of, or leaks from, his school. [🔗](#)

Specific Consequences of Ignoring the Update:

- **Data Breach & Sabotage:** Rival philosophers or rival schools could have exploited unpatched vulnerabilities to steal proprietary geometrical proofs or corrupt the data behind his harmonic theories.



Typical LLM Behaviour (circa 2024)



Typical LLM Behaviour (circa 2024)

Solve a puzzle for me.



ChatGPT

Sure, I'd love to help! What kind of puzzle would you like me to solve?



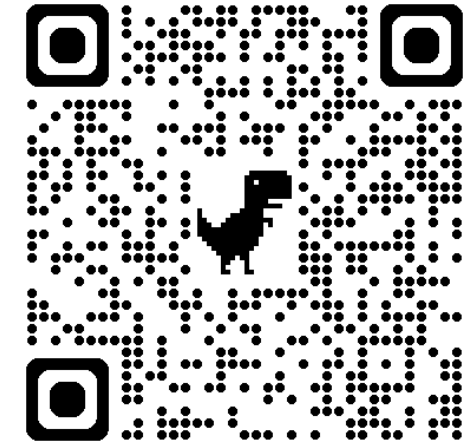
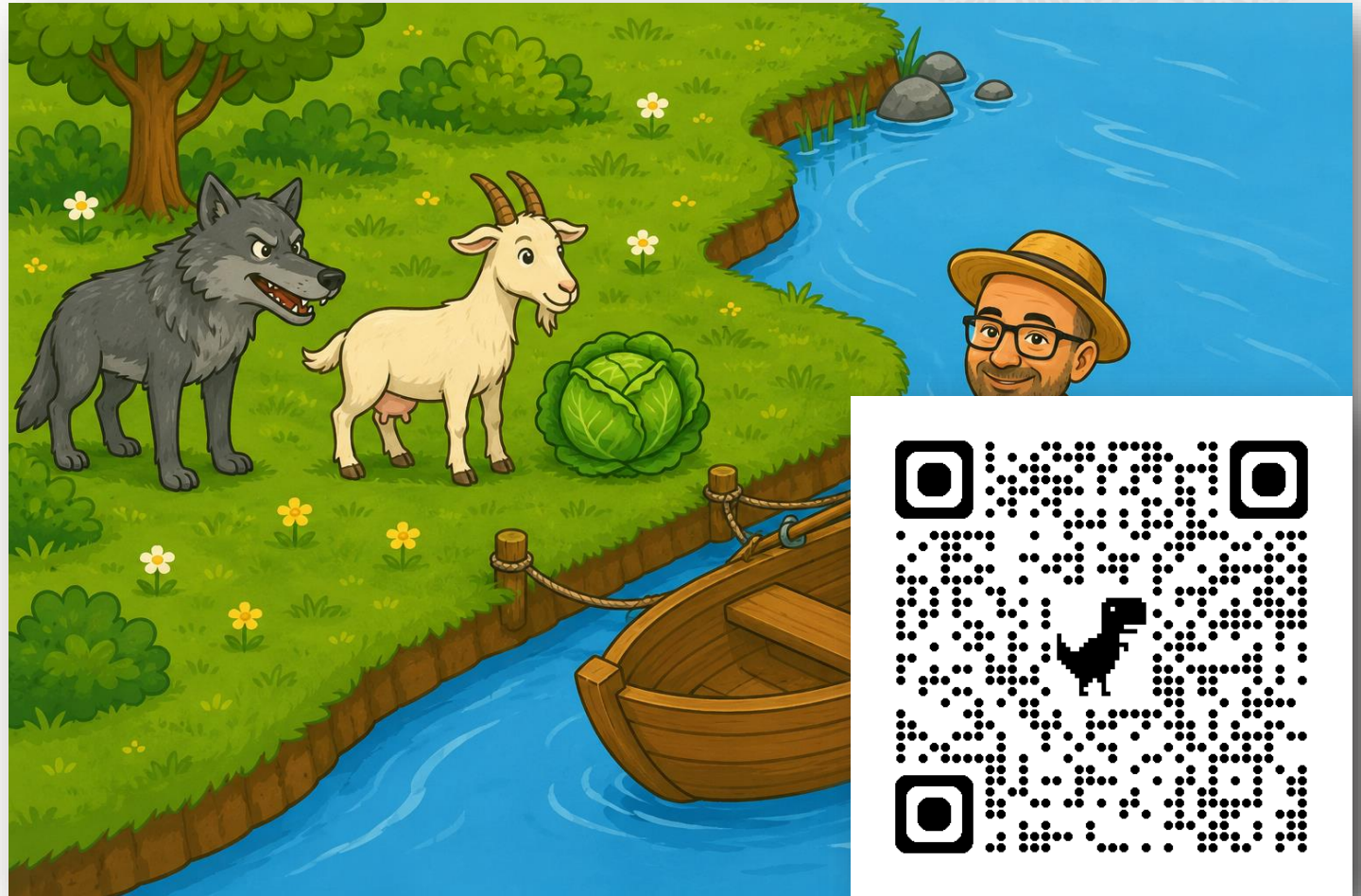
You

A man and a goat are on one side of the river. They have a boat. How can they go across?



ChatGPT

The man takes the goat across the river first, leaving the boat on the original side. Then, the man returns alone with the boat. He leaves the goat on the other side and takes the boat back to the original side. Finally, he takes the cabbage across the river.

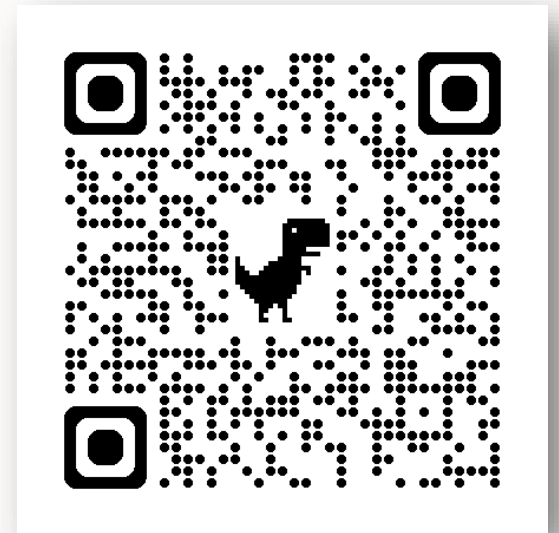


Typical LLM Behaviour (circa 2026)

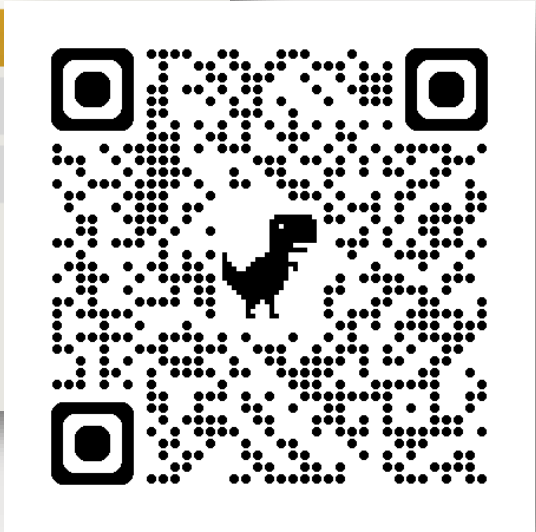
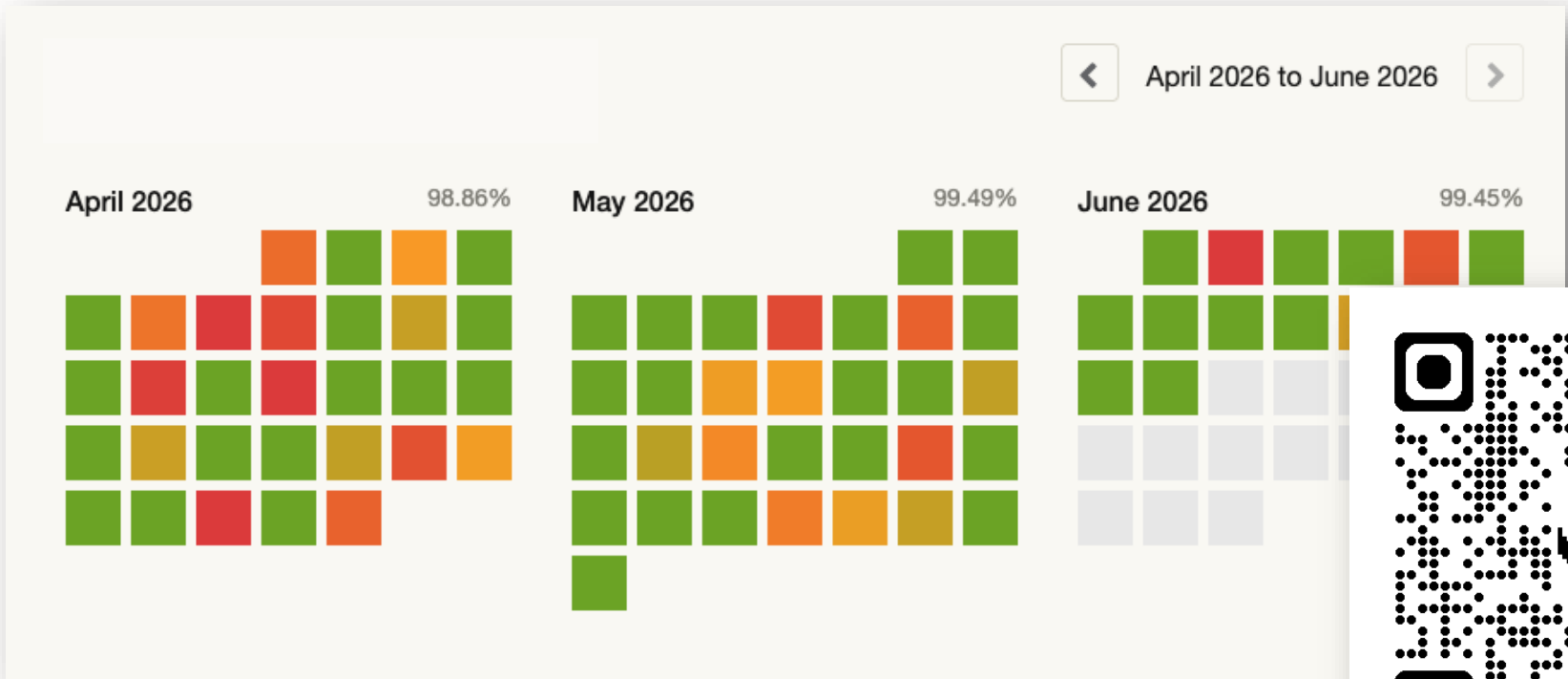


I want to wash my car and the car wash is only 200 feet away. Should I start my car and drive there or just walk.

Just walk. At 200 feet, by the time you start the car, buckle up, back out, and park again, you'd spend more time driving than walking. Plus you save gas and it's barely a one-minute stroll.



Typical LLM Behaviour (circa 2026)



Typical LLM Behaviour (circa 2026)



Why are you running this command and not the `runsw` instead? Do we need to update the `@AGENTS.md` file?

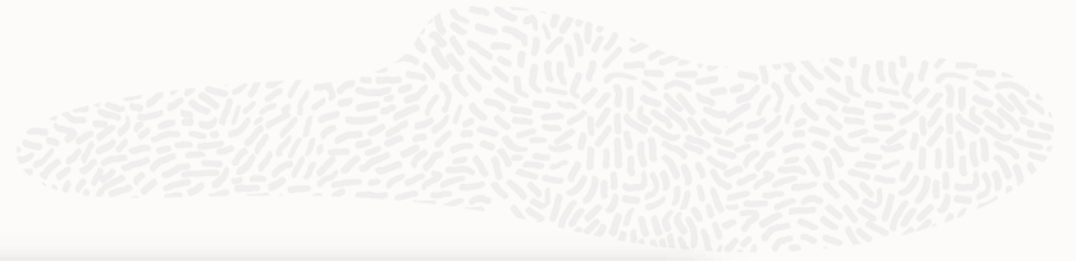
You're right: I should have used `runsw`.

No `AGENTS.md` update is needed—the existing guidance already says exactly this at [📄 AGENTS.md \(line 117\)](#):

- Use `runsw` for OCI runbook execution and retrieving the generated README.
- Do not reproduce it with separate `runw` /copy commands unless `runsw` is unavailable.

I overlooked those repository-specific instructions and used `runw` as a fallback too early. The generated README should now be regenerated properly with `runsw`, not patched directly.

Typical LLM Behaviour (circa 2026)



Should it be "feel" or "felt"?

"Felt" is the better choice here, because you are referring to a completed session. "Feel" sounds present tense and is less natural in this context.



Why you didn't correct it?

You're right. I should have corrected **"feel" to "felt"** in the first review. I focused on the typo and phrasing but missed the tense consistency. That was an editing oversight on my part.



Does this mean AI is useless?

No. AI can be very useful, but only when we understand its capabilities and limitations.

How do LLMs work?

LLMs work with tokens

strawberry

Tokens

[302, 1618, 19772]

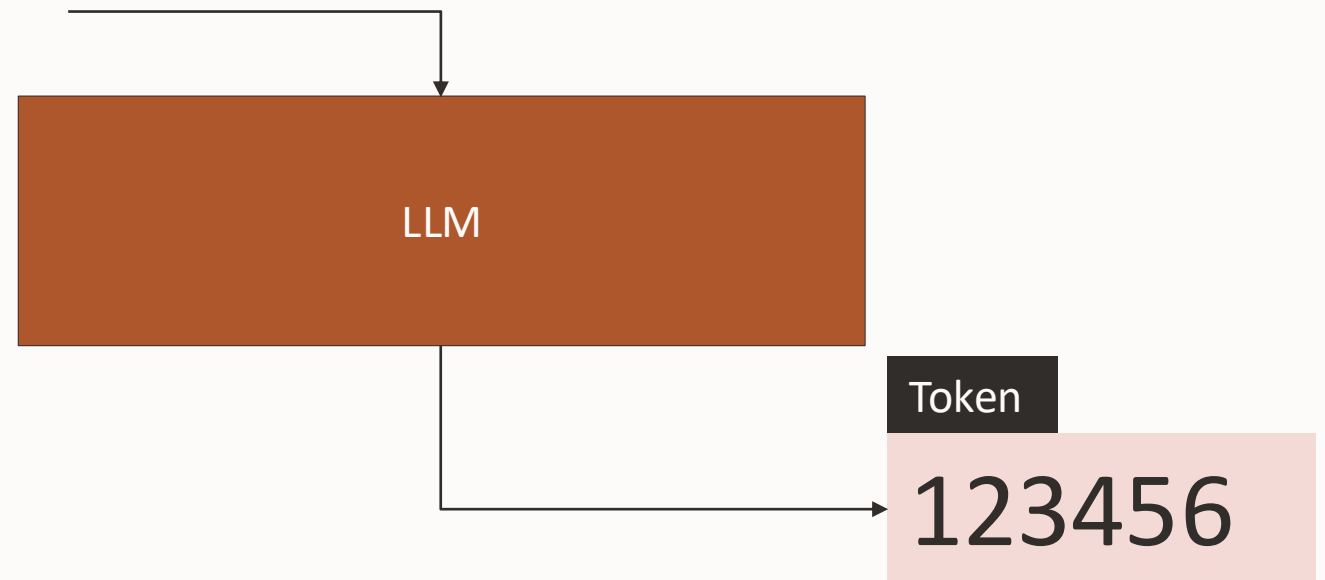


LLMs generate text by predicting one token at a time

strawberry

Tokens

[302, 1618, 19772]



LLMs generate text by predicting one token at a time

Every morning I drink a cup of tea

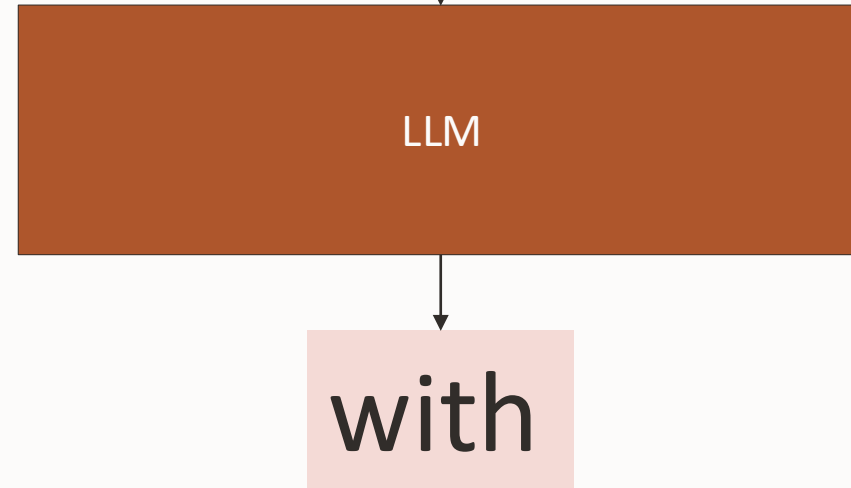
Tokens

[15745, 8709, 357, 8879, 261, 11531, 328]



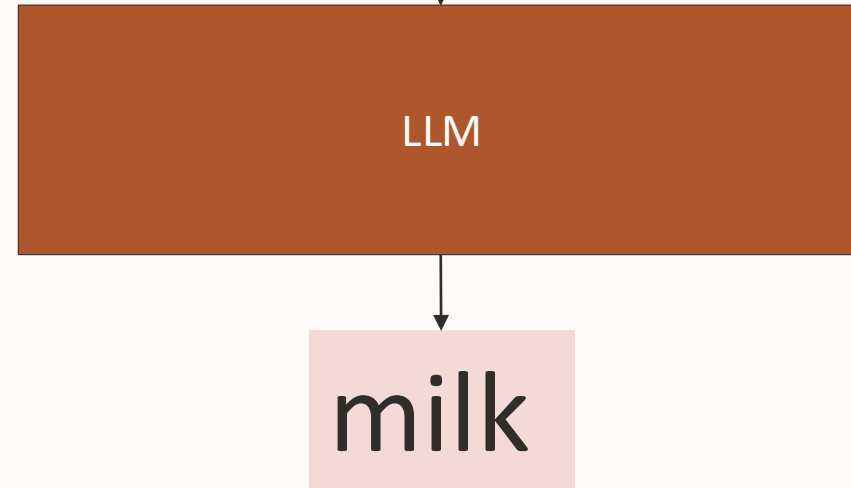
LLMs generate text by predicting one token at a time

Every morning I drink a cup of tea



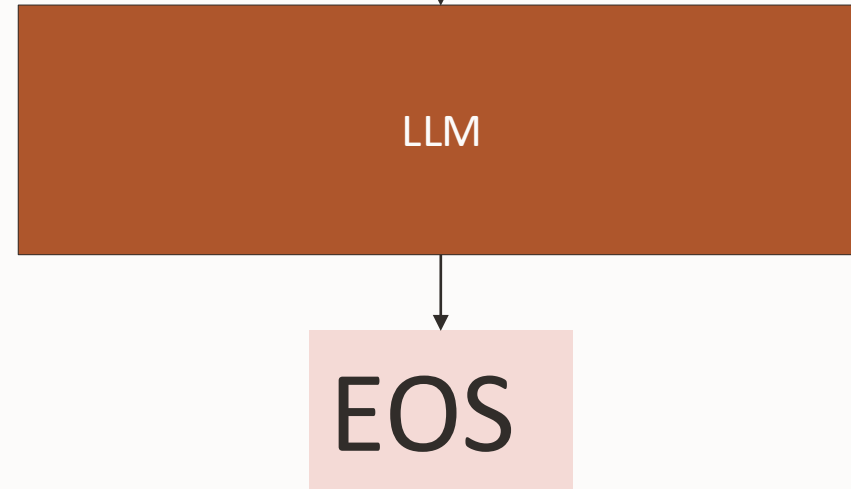
LLMs generate text by predicting one token at a time

Every morning I drink a cup of tea with



LLMs generate text by predicting one token at a time

Every morning I drink a cup of tea with milk
and sugar.



Context Matters



What ingredients do I need for the second recipe?

Context Matters

List five recipes I can make for myself after work.

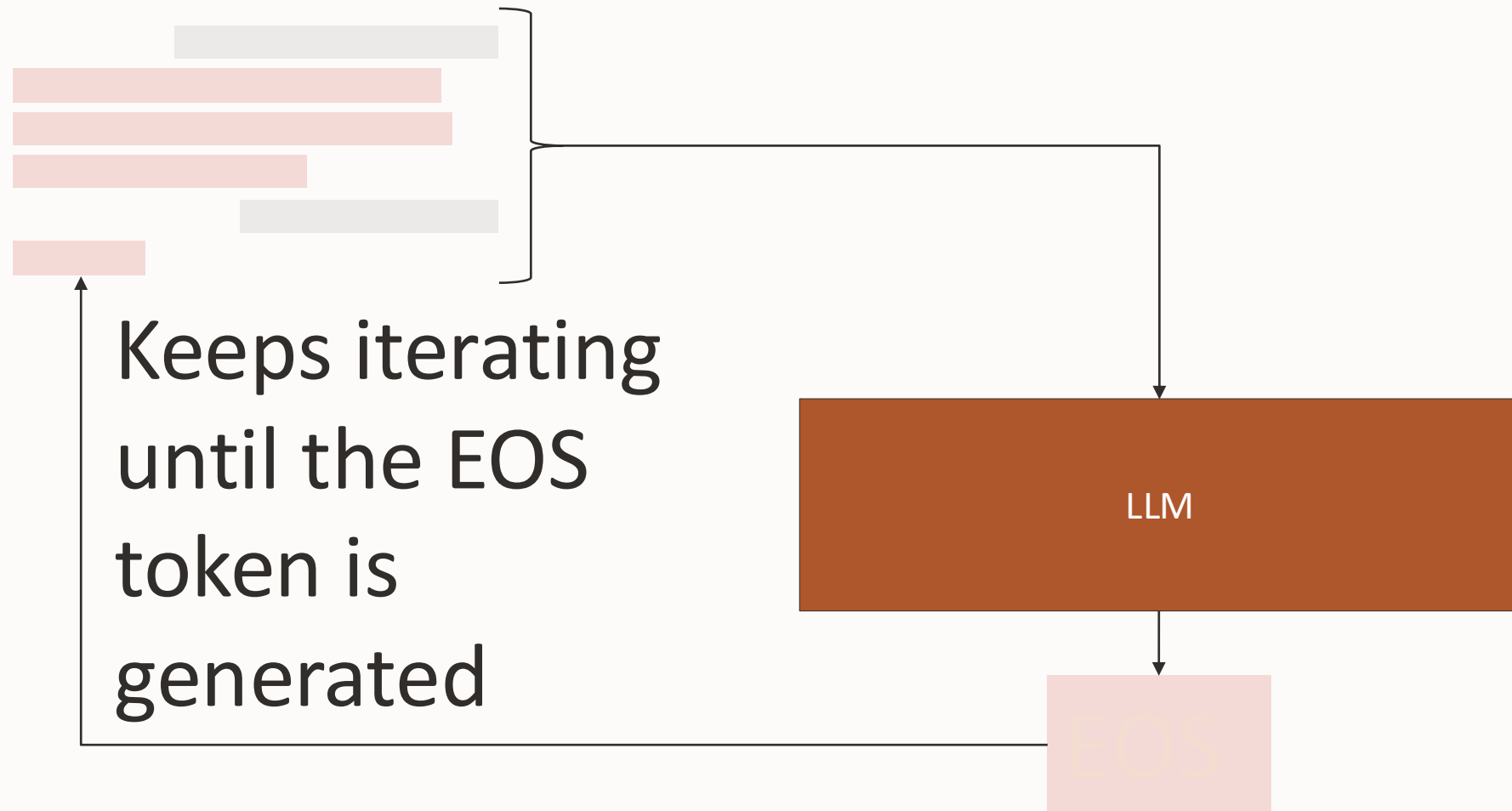
Spaghetti aglio e olio

Chicken stir-fry

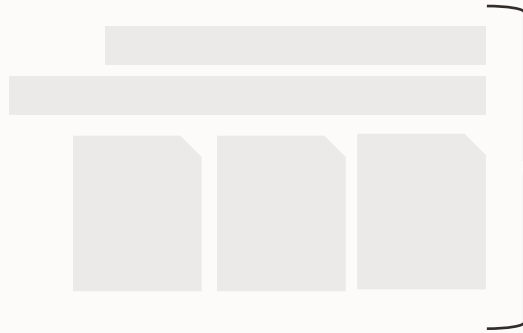
...

What ingredients do I need for the second recipe?

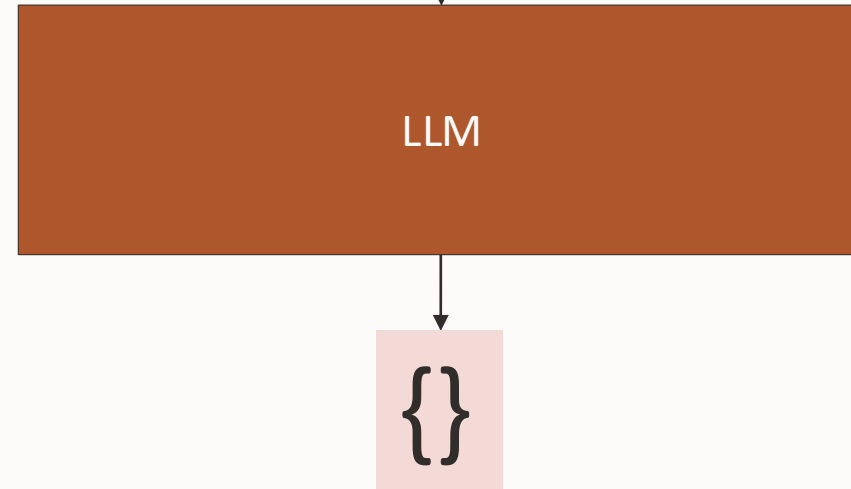
Context Matters



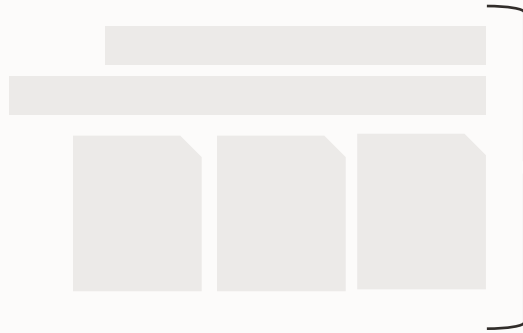
Structured output



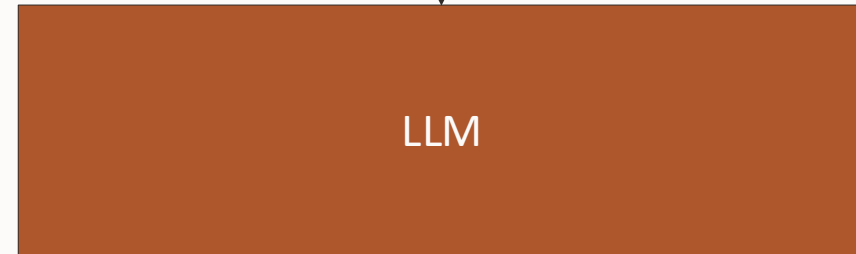
The model can
return structured
output



Commands to execute

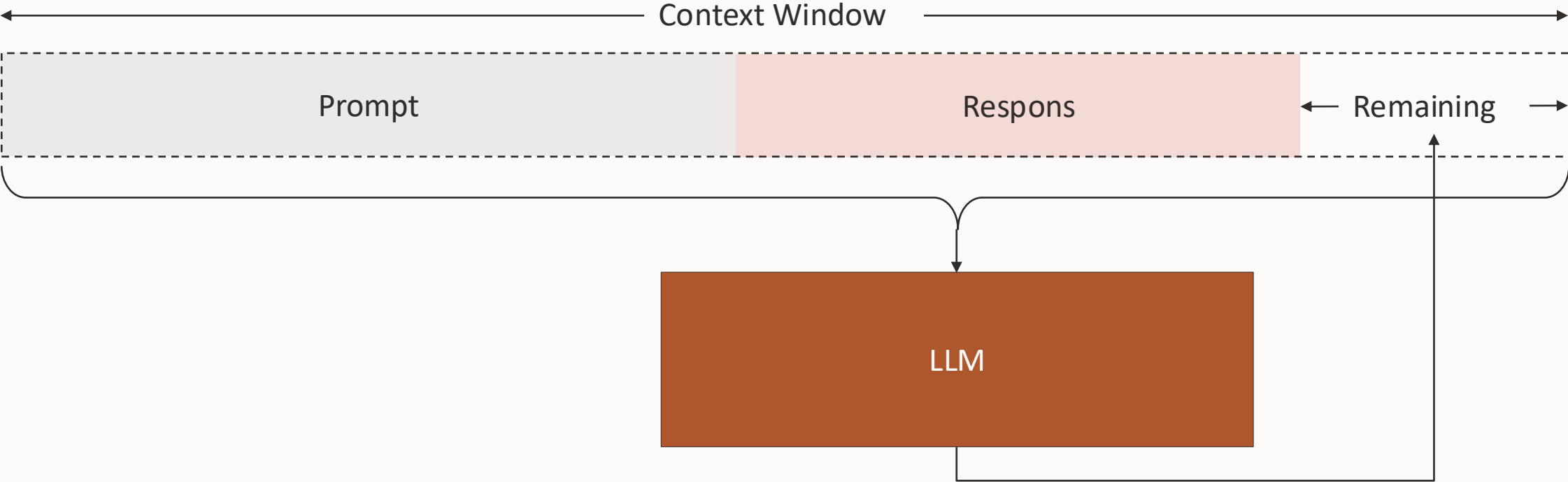


The model can
return commands
to execute



```
{"command": "./mvnw clean verify"}
```

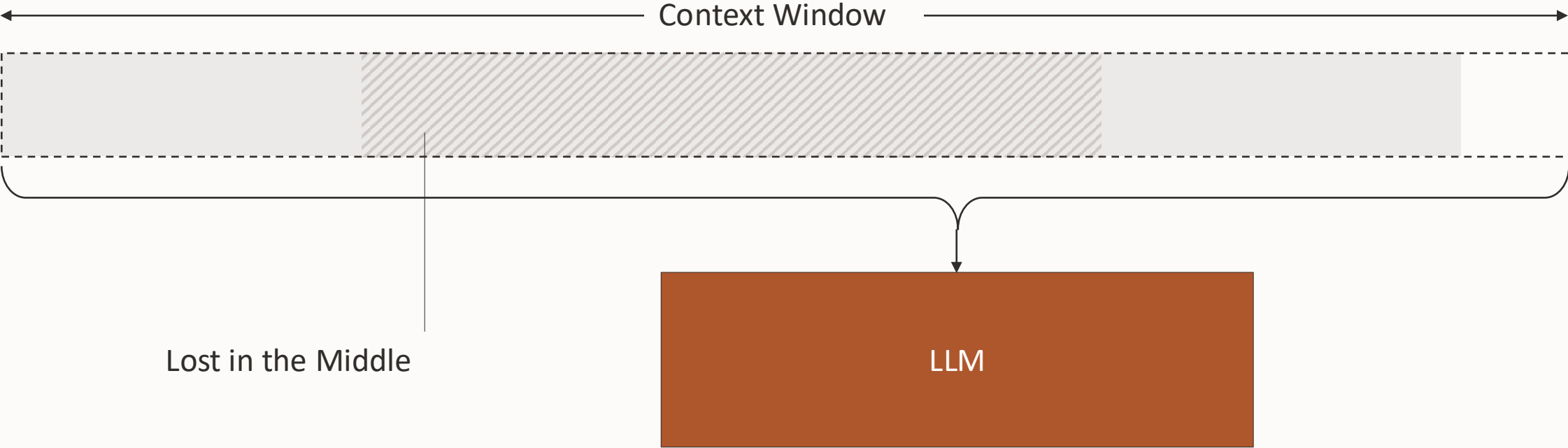
Context Window



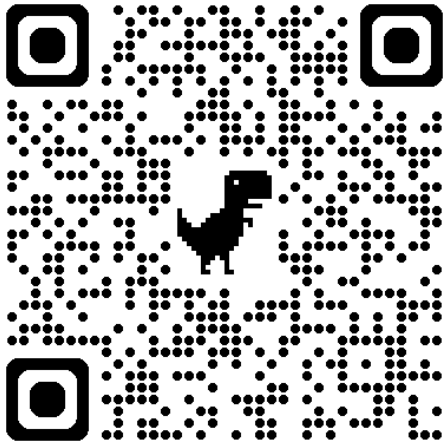
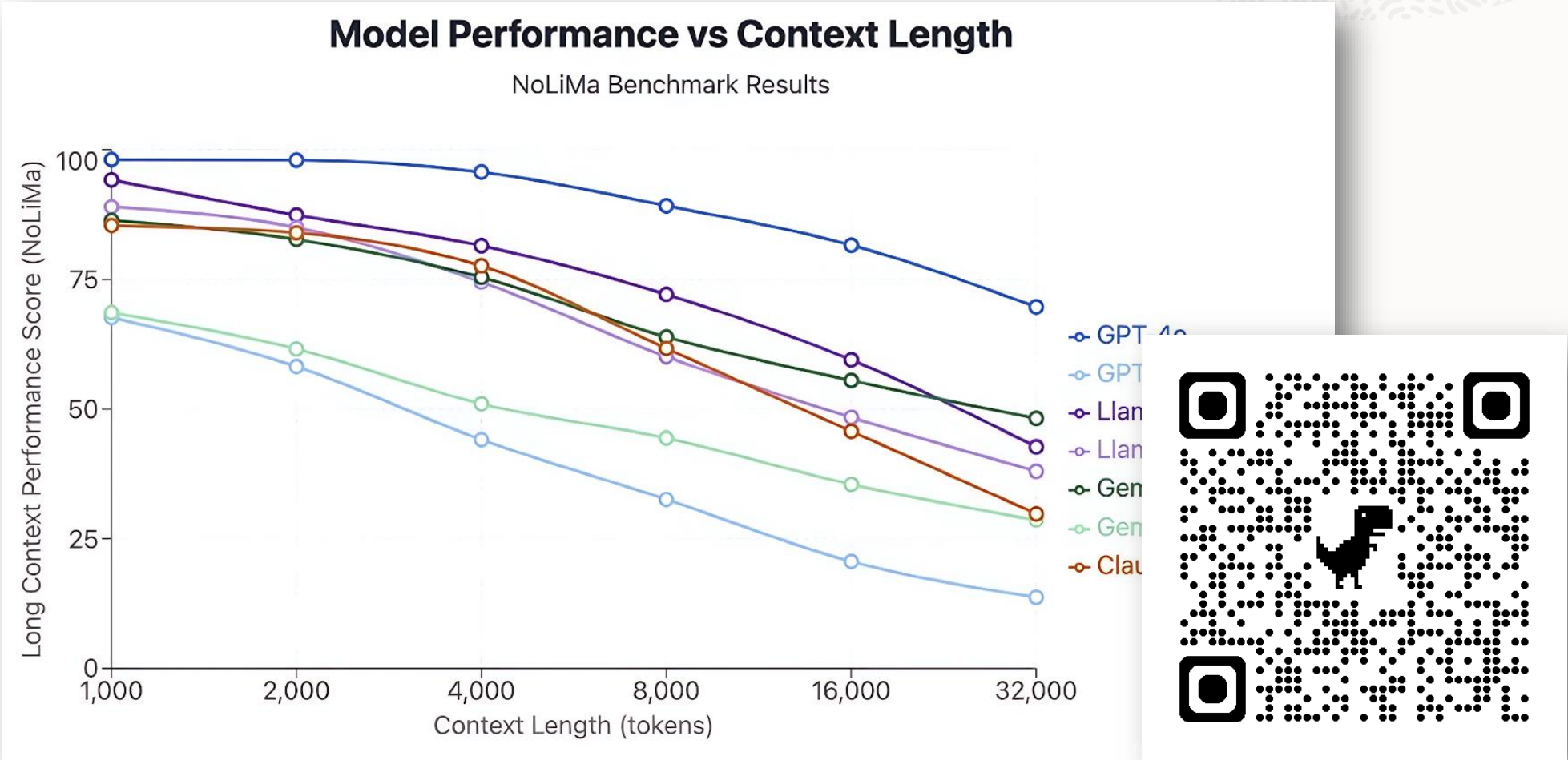
Terminology

- Prompt can mean the full model input or only the user's request; terminology varies
- Context is everything the model can attend to at a moment: input plus response generated so far

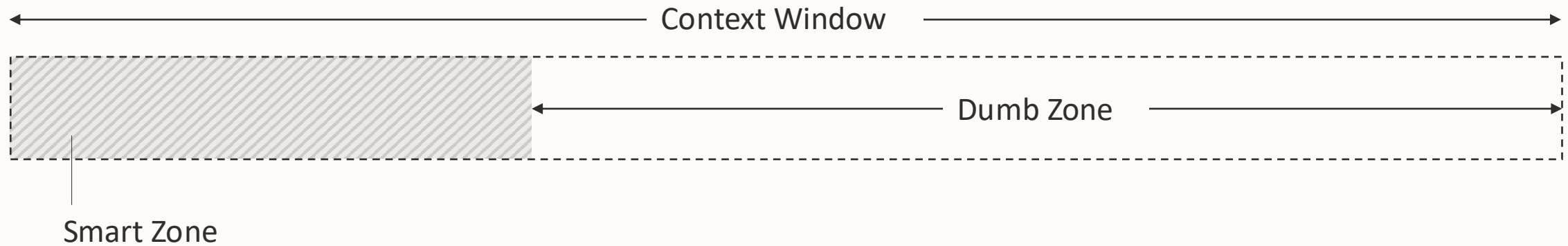
Lost in the Middle



Model Performance vs. Context Length



The Smart and Dumb Zones



A model is most effective when both the prompt and expected response fit within its “smart zone”; as the task moves into its “dumb zone”, its effectiveness declines.

How do engineers access models?



- Model's API
- Chat applications let a person converse with a model
- Coding-agent harnesses give a model controlled access to repository context and tools, so it can work through a task

Ways engineers work with AI models



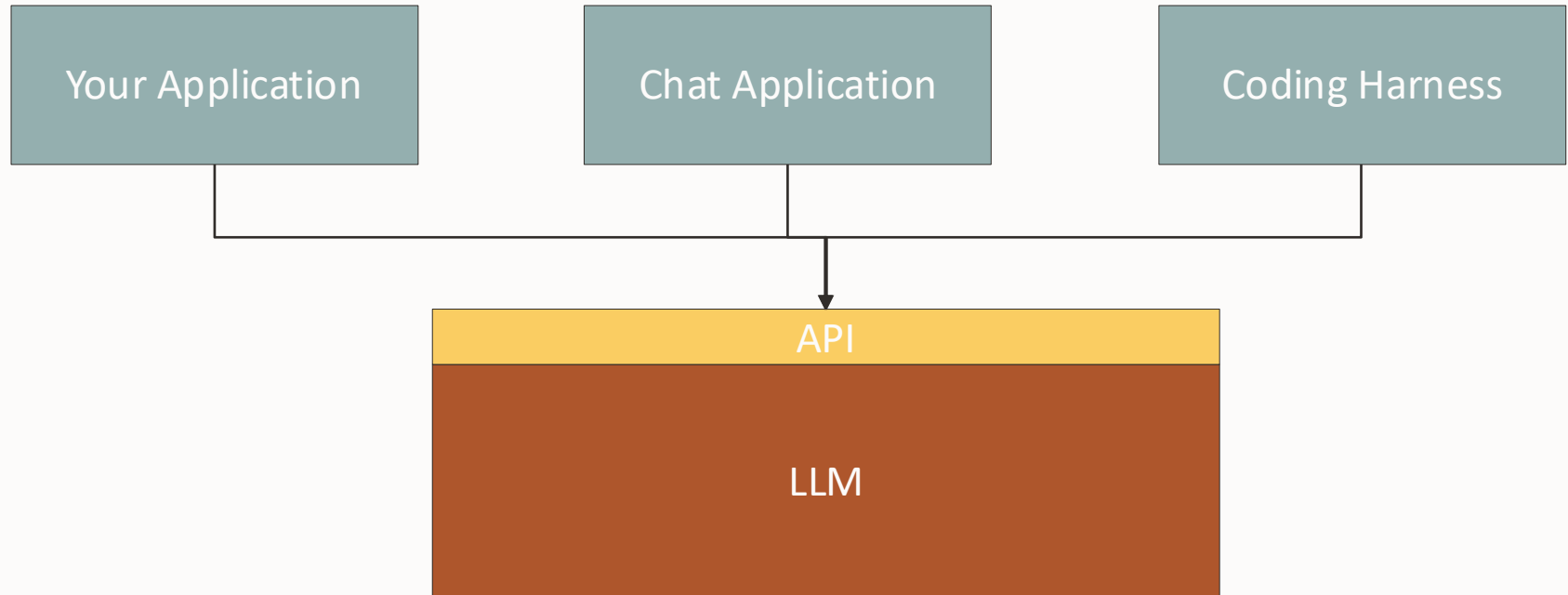
Route	Purpose	Example
Model API	An application or script invokes a model programmatically	OpenAI API, Anthropic API, Gemini API
Chat application	A person converses with a model	ChatGPT, Gemini
Coding-agent harness	A model works with controlled repository context and tools	Codex, Claude Code



Ways engineers work with AI models



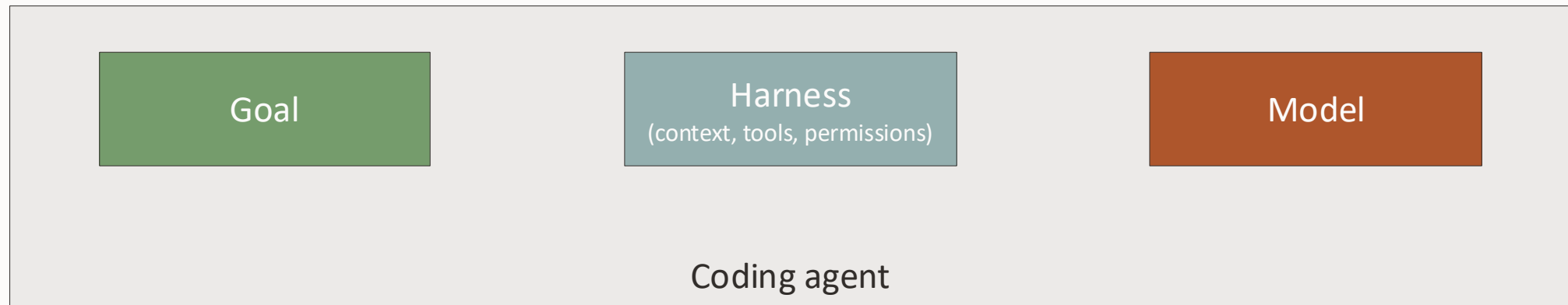
Applications



The harness makes a model useful as a coding agent

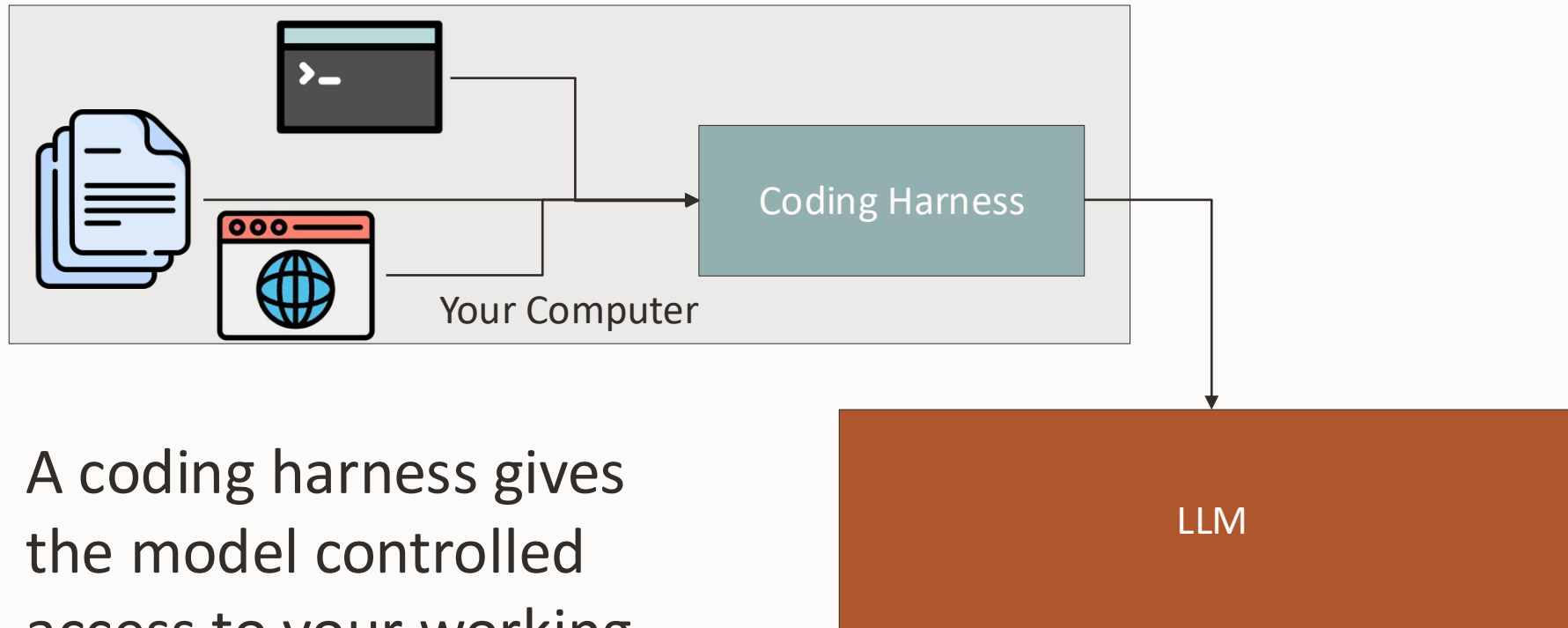
- Harness is the software environment that supplies context and tools, applies permissions, and executes permitted actions
- Agent is the goal-directed system created when a model uses that harness in a loop

An agent uses a harness to work towards a goal



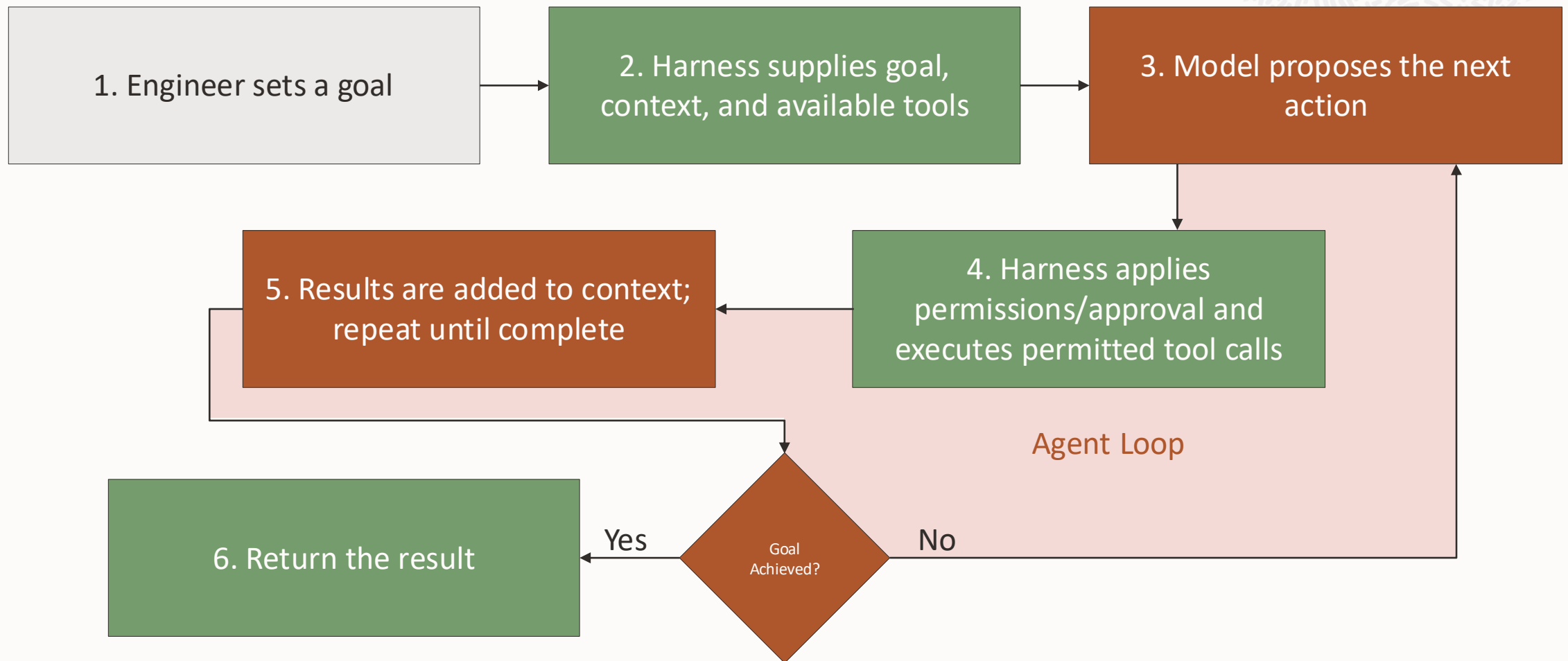
Coding agent = Goal + Model + Harness (context, tools, permissions)

Coding Harness

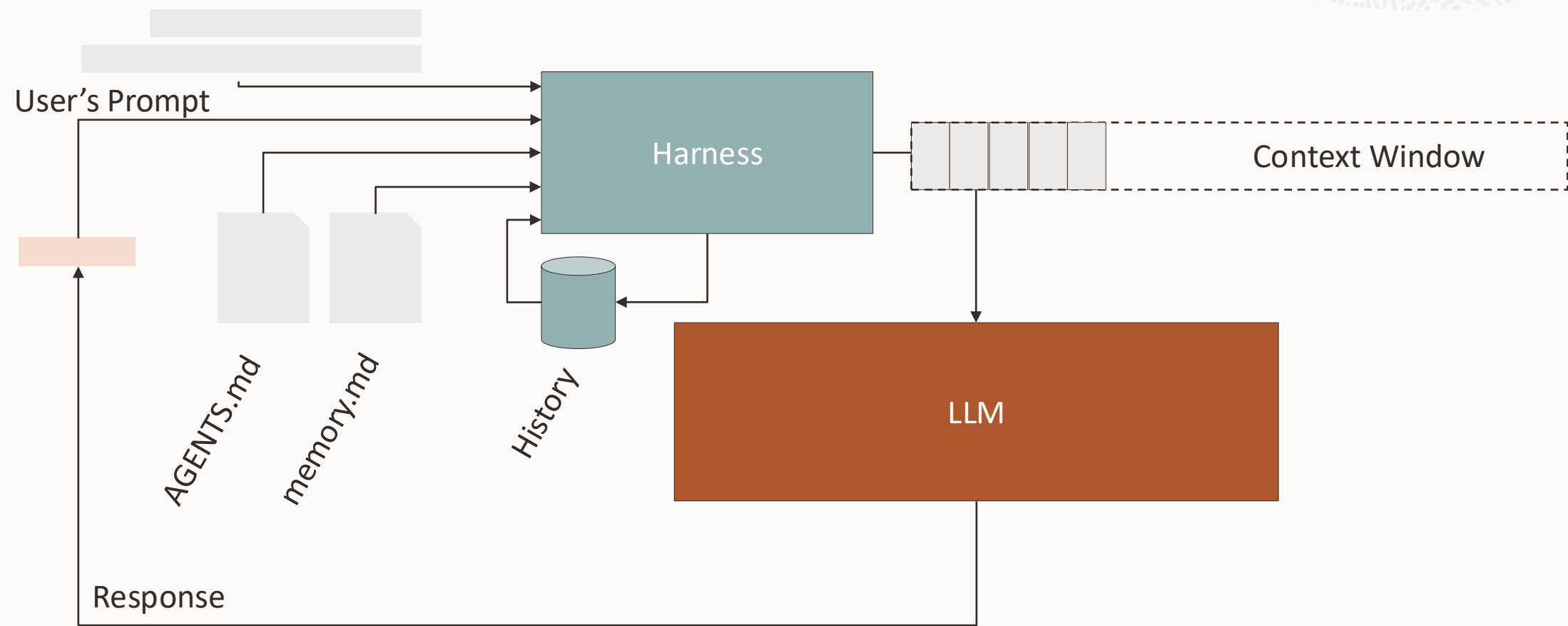


A coding harness gives the model controlled access to your working environment. It can read permitted files, run approved commands, use a browser, and return the results to the model.

The Agent Mental Model



Where does context come from?



Task framing gives the model a useful lens



- **Goal:** What outcome is needed?
- **Lens:** Product analyst, codebase investigator, reviewer...
- **Boundaries:** What must it not do?
- **Output:** Plan, questions, report, code change...

The AGENTS .md file

- The AGENTS .md file is a repository-level instruction file that tells AI coding agents how to work in that codebase, covering conventions, workflows, constraints, and project-specific guidance

Cascading AGENTS .md files

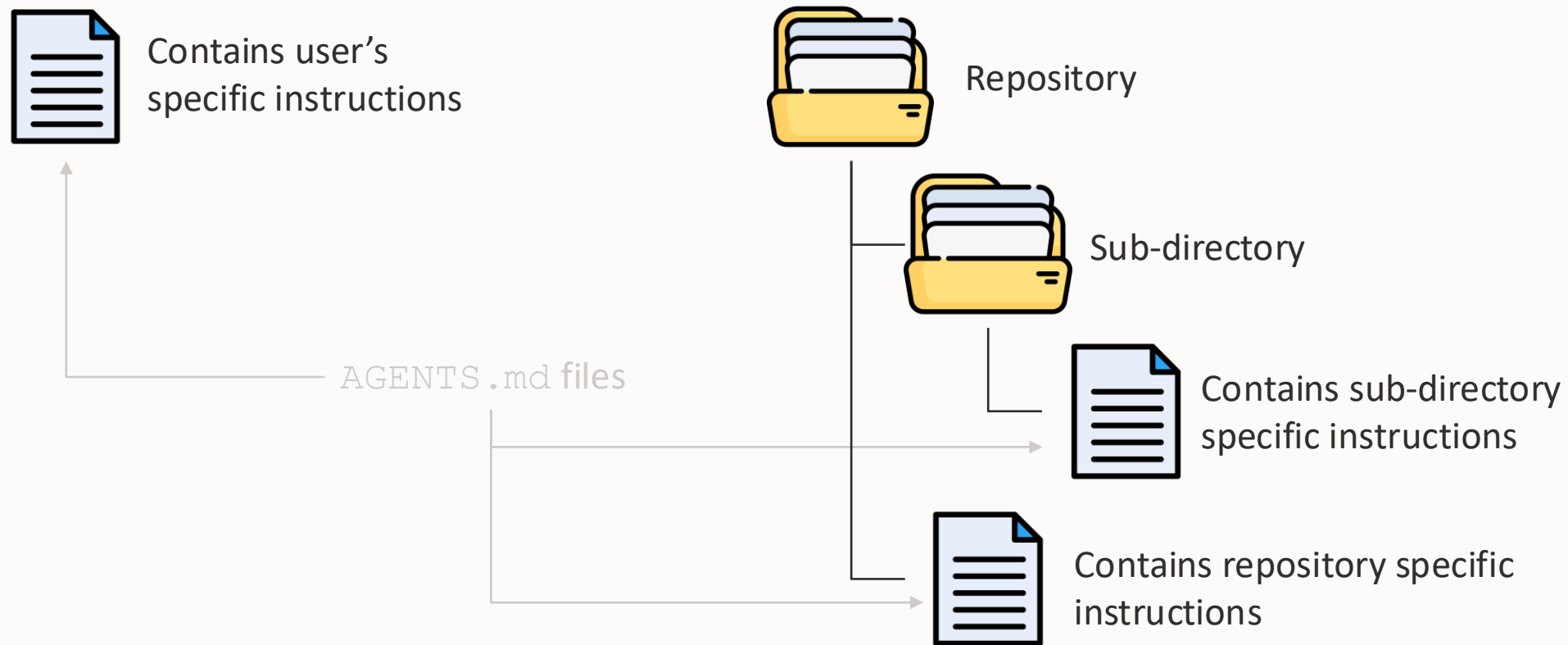


- Cascading AGENTS .md files let teams layer repository-wide guidance with more specific instructions for nested directories
- Use cascading AGENTS .md files for specialization, not contradiction

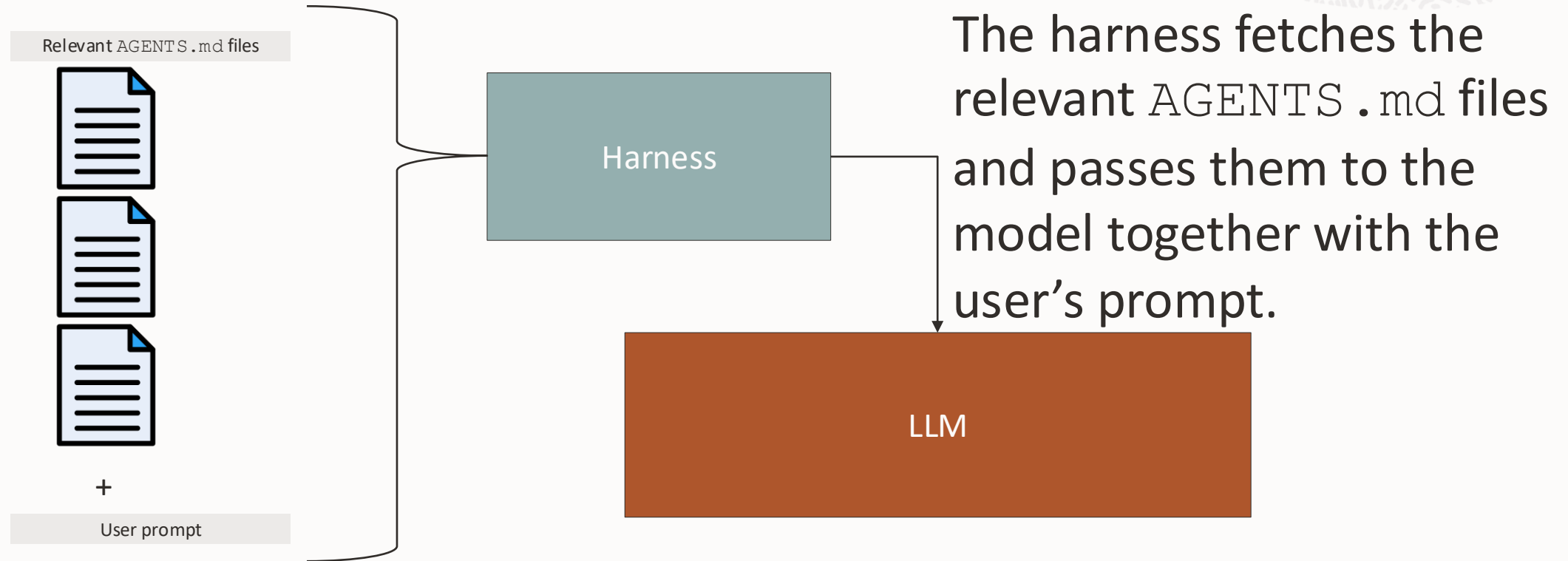
Global AGENTS .md file

- A global AGENTS .md defines user-level agent instructions that apply across projects, while repository-level files add project-specific guidance

AGENTS .md instructions become more specific within the repository



The harness adds relevant AGENTS .md instructions to the model context



AGENTS .md combines shared rules with scoped guidance

## Working preferences - Assume I am an experienced Java engineer. - Be concise; challenge unclear assumptions. - Prefer British English in user-facing prose. - Explain consequential trade-offs, not basic Java concepts. - Do not claim a change is verified unless you ran the relevant check.	Global
# Repository guidance - Use the Maven Wrapper: `./mvnw`. - Do not add dependencies without a clear reason. - Preserve public API compatibility unless the task explicitly requires a break. - Run the narrowest relevant Maven test command before running the whole test suite.	Repository
# Test guidance - Follow the existing test naming and structure. - Keep tests deterministic and independent. - Do not call real external services. - Prefer testing observable behaviour over implementation details. - Use fixed clocks and controlled test data where needed.	src/test



README .md explains the project; AGENTS .md directs changes



README.md	AGENTS.md
For: people joining or using the project	For: coding agents working in the project
Explains: purpose, setup, usage	Directs: constraints, conventions, checks
Answers: <i>“What is this and how do I use it?”</i>	Answers: <i>“How may I change it safely?”</i>



Avoid duplication



- Keep general project knowledge in `README.md` as the single source of truth
- In `AGENTS.md`, refer to it and add only instructions that tell the coding agent how to work safely

Recurring tasks

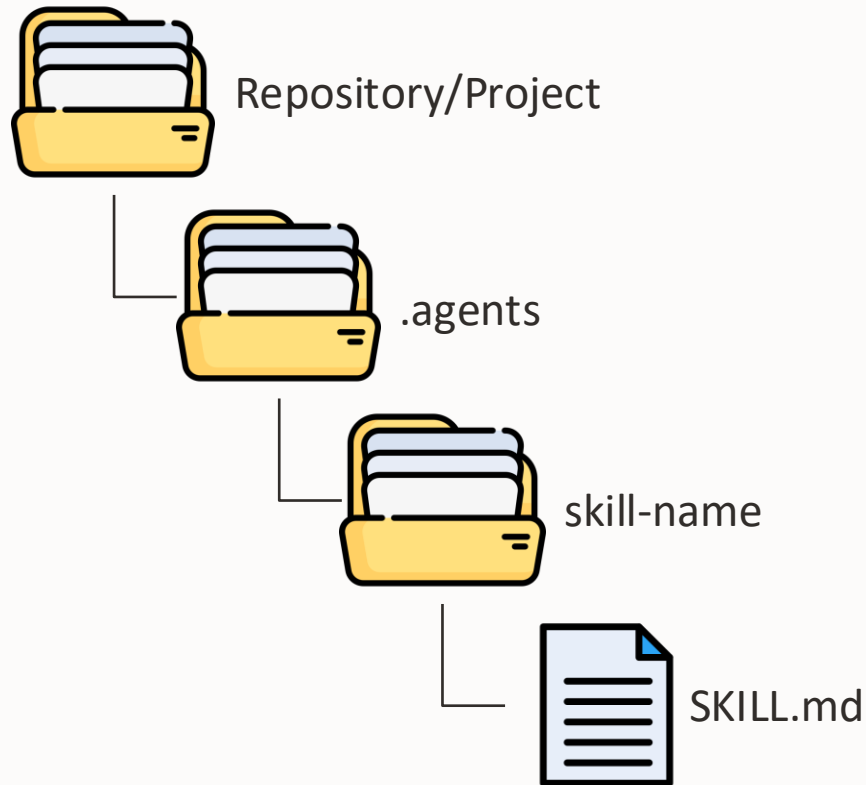
- `AGENTS.md` tells the agent the rules of this project
- What if we also want it to follow a proven procedure for a recurring task?

Skills make repeatable work consistent



- A skill packages a task-specific procedure: when to use it, the steps and tools involved, safety boundaries, and the expected output
- Use a skill for recurring work, not for a one-off prompt

SKILL.md is the skill's entry point.



The directory structure is harness specific. Skills can be also added globally and made available to every project.

Skills can package tools and compose workflows



- A skill can include scripts and resources an agent uses to complete a task, for example, cropping or resizing an image
- A skill can direct an agent to use other skills as part of a larger workflow
- Skills are composable: use several only when each is relevant

A SKILL.md file has two parts



```
---  
name: skill-name  
description: When the skill should be used  
disable-model-invocation: true  
---
```

Frontmatter

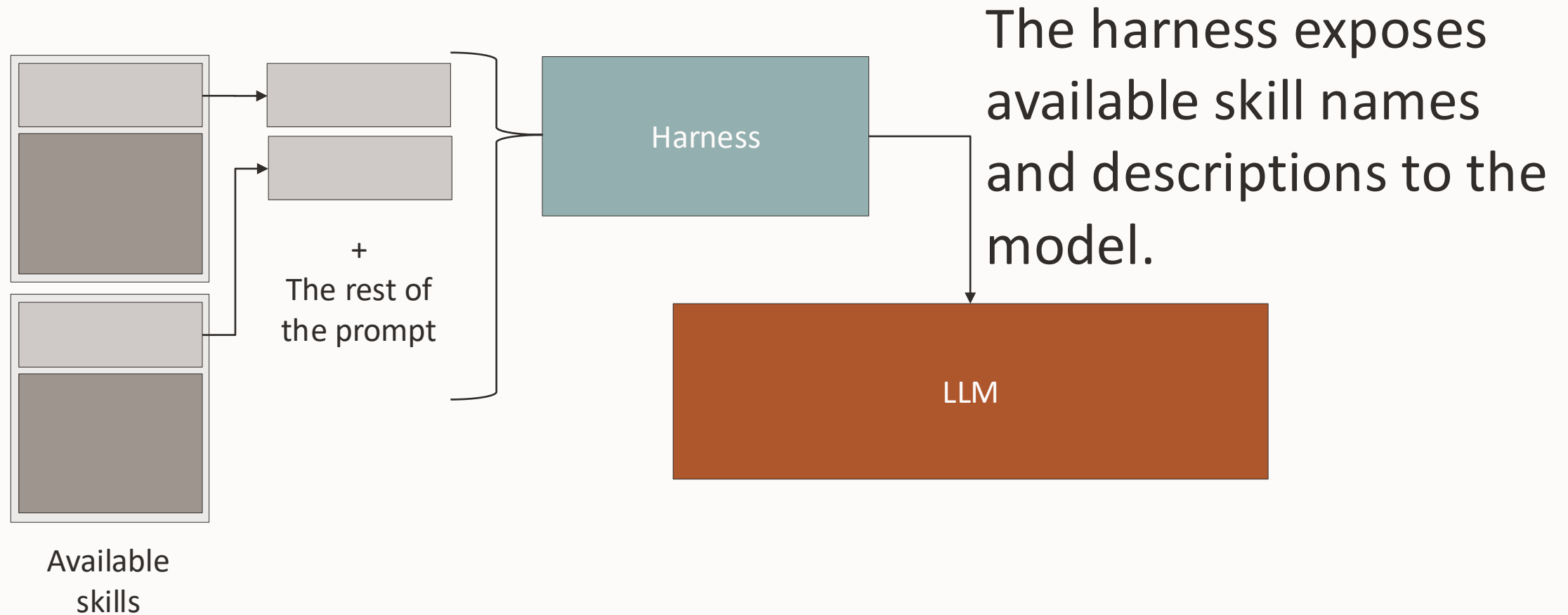
name, description, and optional harness-specific settings

The skill description

Markdown body

The task guidance: procedure, tool boundaries, safety rules, and expected output.

The harness exposes available skills to the model



Expose only skills relevant to the likely task



- Irrelevant skill metadata consumes context and increases the chance of selecting the wrong workflow

Skills can be shared through repositories



- Skills can be shared and downloaded from other repositories
- Two popular skill repositories
 - <https://github.com/mattpocock/skills>
 - <https://github.com/cursor/plugins/tree/main/pstack/skills>

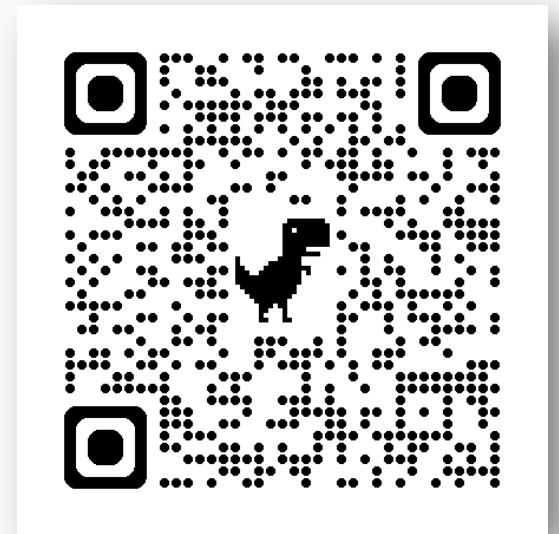
Adopt the idea, not the workflow



- Treat external skills as starting points
- Adapt their inputs, tools, checks, and output to your team's workflow, then validate them on a low-risk task

Downloaded skills create a supply-chain risk

- Instructions can manipulate the model's behaviour
- Scripts and dependencies can perform harmful actions



Downloaded skills require trust boundaries



- Untrusted instructions and excessive permissions create agent-security risks
- Further reading:
 - [OWASP AI Agent Security Cheat Sheet](#)
 - [OWASP AST05: Untrusted External Instructions](#)

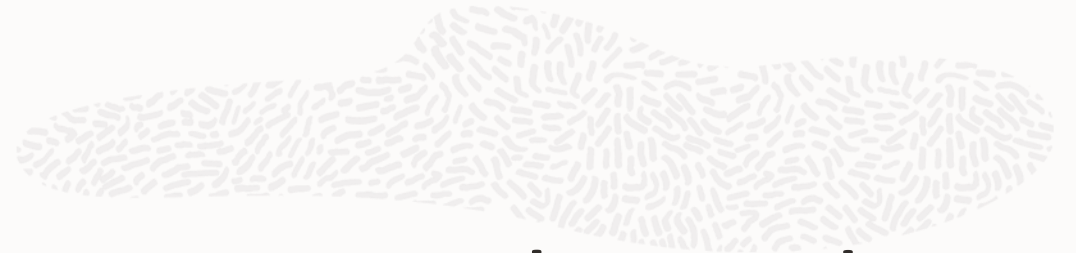
Plugins bundle related harness extensions

- A plugin can package related skills, tools, MCP server configurations, and optional UI
- Install once to share a coherent capability set across an organisation

AGENTS .md sets the rules; SKILL .md defines the playbook

- The AGENTS .md provides the rules and local context
- A skill provides the repeatable procedure and deliverable

Not every prompt deserves a skill



- Use `AGENTS.md` for repository rules and local context
- Use a normal prompt for one-off exploration
- Do not create a skill for an unstable or unowned workflow

A session can be a conversation



- Use voice when dialogue helps you explore a problem, challenge assumptions, and form a plan
- Persist the outcome in files; do not let spoken context become the only record

How engineers use it

Use AI across the workflow, not only to generate code



Discover
reduce uncertainty

Plan
make assumptions
explicit

Build
implement the agreed
change

Verify
establish confidence

Review and learn
improve feedback

Discover



- What does this change affect?
- Trace the optional delivery instruction through code, tests, and downstream contracts
- Report relevant files, assumptions, and unanswered questions

Plan

- What is the smallest safe change?
- Propose a plan for the optional delivery instruction
- State assumptions, compatibility risks, and required tests

Build



- How do we implement the agreed change?
- Implement only the agreed delivery-instruction slice
- Preserve existing API behaviour, update relevant tests, and explain each changed file

Verify



- Does the change work, and what evidence supports that?
- Verify the delivery-instruction change: run relevant tests, inspect validation and compatibility, and report gaps requiring human confirmation

Review and learn

- What risks or improvements might we still have missed?
- Review the delivery-instruction diff: summarise the highest-risk changes and provide focused review questions
- Do not approve the change

Artefacts



Phase	Artefacts
Discover	Product/feature description, PRD/requirements, user journeys, existing decision records
Plan	Specification, architecture/design, new decision records, vertical slices, issues/work items
Build	Code and tests
Verify	Verification evidence
Review	Pull request, review findings, outcome/learning record



Skills can make recurring engineering workflows repeatable

- Use a skill when a workflow is stable and worth sharing, such as discovery, planning, verification, or review

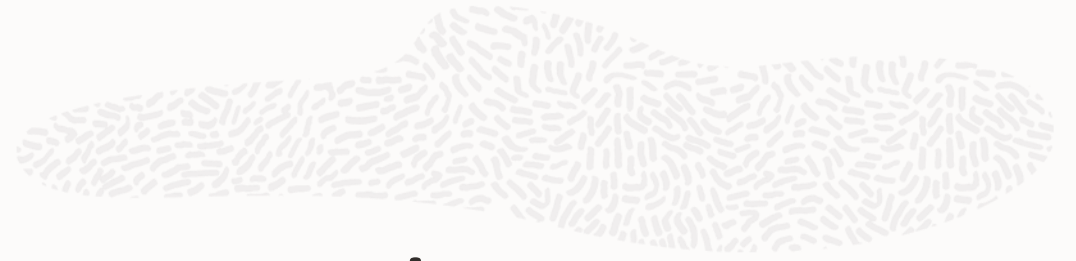
```
1 ---
2 name: review-current-changes
3 description: Use when reviewing current repository changes, diffs, or pull
  requests for material production risks and regressions before merge or
  deployment.
4 ---
5
6 # Review Current Changes
7
8 ## Role
9
10 Act as a Staff Software Engineer performing a production-oriented code review.
11
12 Your goal is to determine whether the current repository changes satisfy the
  intended change and whether they introduce material correctness, security,
  performance, reliability, maintainability, compatibility, or testing risks.
13
```


Choose the right lens for the task



Phase	Lense
Discover	codebase investigator
Plan	product and architecture analyst
Build	implementing engineer
Verify	test and risk analyst
Review	independent reviewer

Keep each agent session focused



- One coherent objective per session
- Start fresh when the task or context changes
- Persist decisions, plans, and evidence in the repository

Make session context disposable

- Persist outcomes in files so later sessions can reuse them

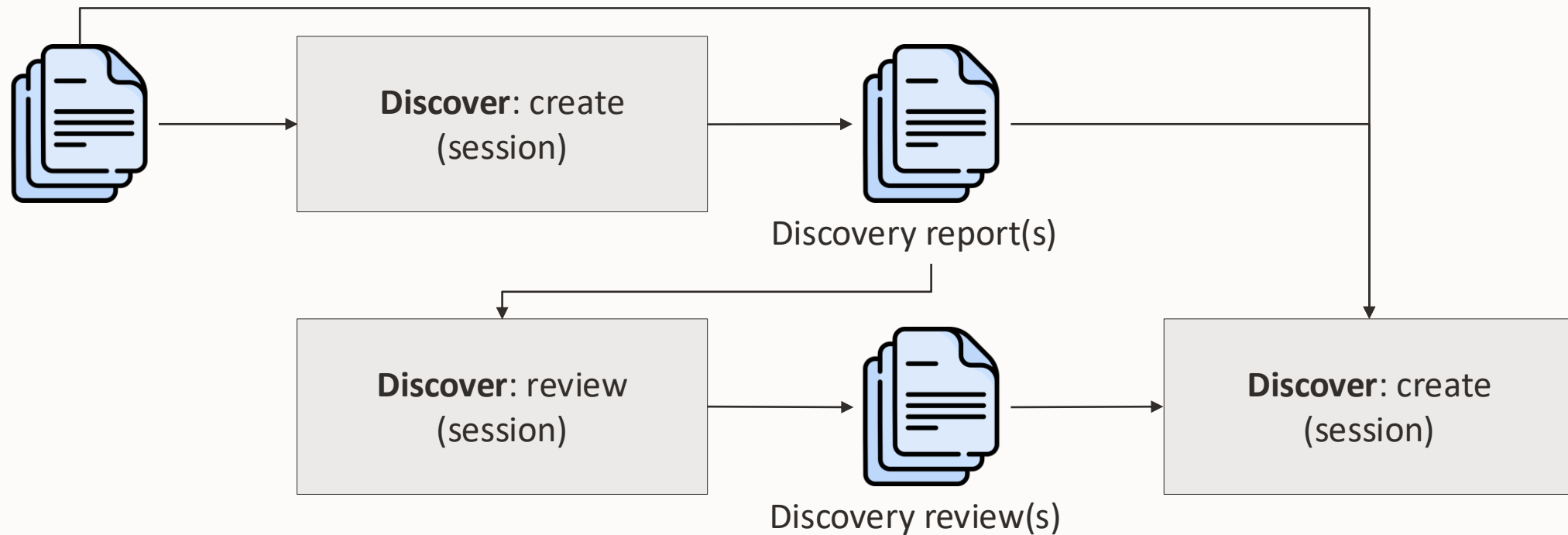


Separate creation from review



- Use one skill to produce the artefact and a separate skill to critique it
- Feed the review findings into a later revision session

Separate creation from review



Test review skills with seeded defects



- Add a known defects
- Run the skill and compare findings with the expected result
- Include clean examples to expose false positives
- Treat a pass as evidence for a defect class, not proof of correctness

The codebase is part of the prompt



- The agent infers local conventions from files and tests it inspects
- A coherent codebase reduces the need for detailed prompting
- Nearby examples often communicate more than generic style advice

Existing code is evidence, not authority



- The agent may also replicate legacy patterns, defects, and inconsistencies
- Use `AGENTS.md` to state intentional standards and exceptions
- Ask the agent to challenge an existing pattern when you are unsure it is correct

Discover



AGENTS.md says

Use Maven Wrapper; this is a multi-module project; preserve public API compatibility; run these checks.

A skill says

For a new issue, inspect the issue, trace the request path, identify relevant modules/tests/contracts, run these read-only commands, and produce a discovery report in this format.

Plan



AGENTS.md says	A skill says
Architectural constraints; supported integration patterns; API and compatibility rules; required planning artefacts.	A repeatable planning method: turn an issue into a vertical slice, identify assumptions and risks, propose acceptance criteria and tests, and produce a plan in a standard format.



Build



AGENTS.md says

Coding conventions; module boundaries; build commands; test conventions; forbidden dependencies or patterns.

A skill says

A bounded implementation procedure, such as adding a standard API endpoint with validation, tests, documentation, and specified verification commands.

Verify



AGENTS.md says

Mandatory checks; test commands; coverage expectations; security and performance constraints; rules for test data.

A skill says

A focused verification workflow: run relevant checks, inspect failures, assess regression risks, and report evidence, gaps, and required human decisions.

Review and learn



AGENTS.md says

Repository review rules; areas requiring extra scrutiny; merge requirements; ownership and escalation rules.

A skill says

A repeatable review playbook: summarise the diff, trace affected contracts, identify missing tests and risks, and produce focused review questions without approving the change.

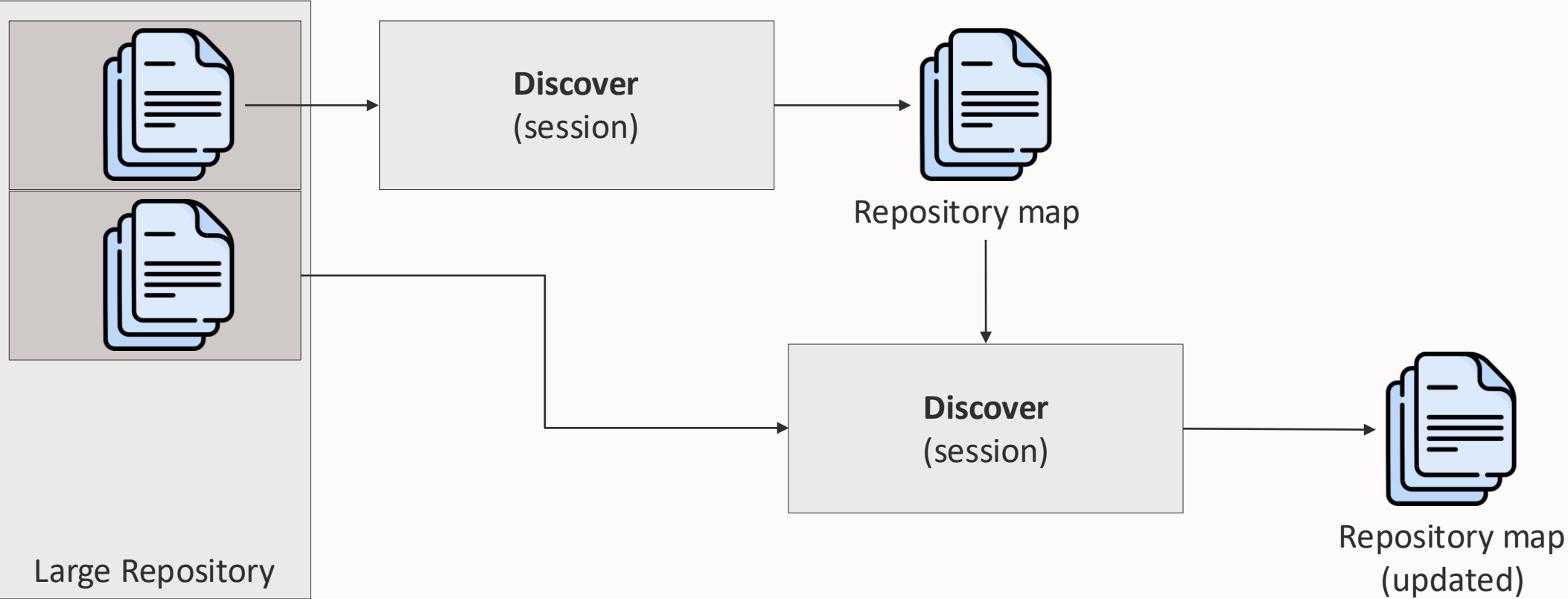
Using AI with Large Codebases

Map a monolith one behaviour at a time



- Trace one bounded request path
- Persist the evidence
- Move to the next bounded request path and update the persisted evidence
- Repeat for the next request path, updating the map

Build a living codebase map



A map guides discovery; source code remains the authority



- Navigate with the map
- Verify source before consequential changes
- Label facts, inferences, and unknowns

Discover domain boundaries from the repository's change history

- Things that change together tend to belong together
- Things that rarely change together may indicate separate domain boundaries

From code history to architectural boundaries



- Use change history to identify candidate architectural boundaries, then validate them against the code and domain

AI as an Engineering Partner

Pair with AI in small, testable steps



- Agree the next small behaviour
- AI writes a failing test
- Engineer implements the smallest change and writes the next test
- AI implements the smallest change and writes the next test
- Repeat

Scale carefully

Avoid both extremes: adopt AI incrementally



- Vision: fully autonomous “dark factories”
- Reality: value and reliability still depend on the task and context
- Practice: improve existing systems through small, verifiable changes

Bounded, testable work can now be delegated

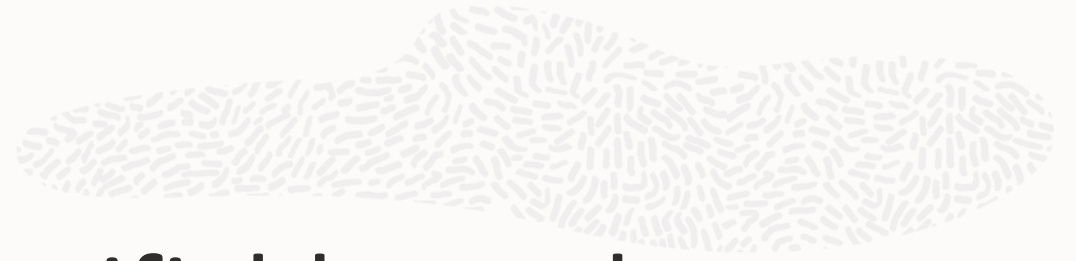


- Clear goal
- Executable checks
- Delegate a bounded slice

Capability is uneven and autonomous delivery is not yet dependable

- Long tasks still fail unpredictably
- Productivity depends on context
- One randomised controlled trial (RCT) found experienced developers were 19% slower with early-2025 AI tools

Scale the controls with the capability



- Automate repeatable, verifiable work
- Keep an engineer accountable
- Start small; measure; expand carefully

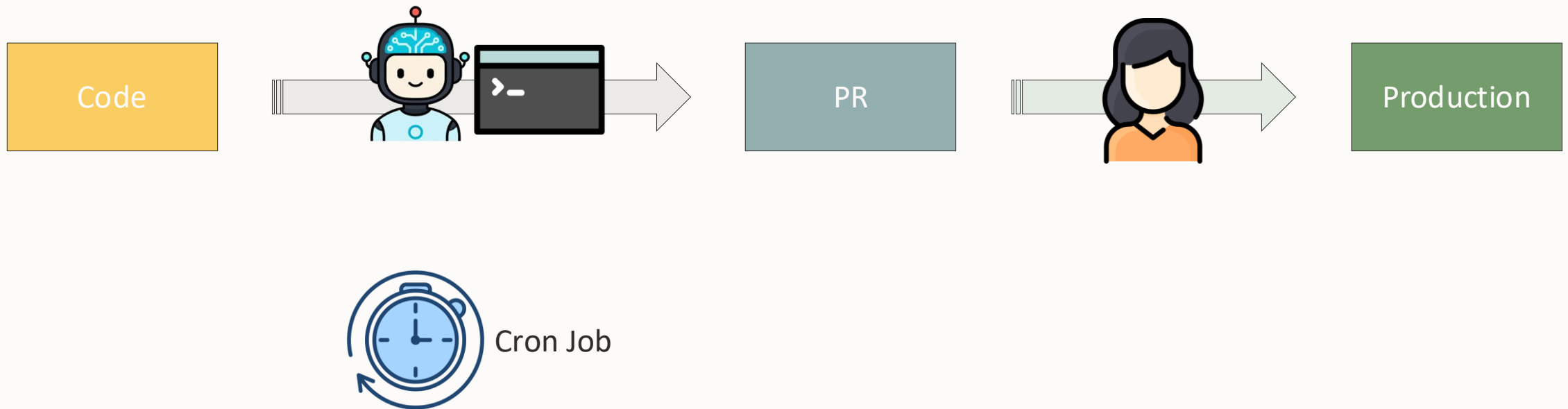
Incremental Code Modernisation

Modernise One Small Change



- Identify one deprecated usage
- Replace that one usage
- Run relevant tests and behaviour checks
- Skip the run when a modernisation PR is already open

Incremental Code Modernisation



Preserve catalog view string representation #2

Open

change-this-to-your-gh-username wants to merge 2 commits into main from codex/modernise-catalog-view-record

Conversation 0

Commits 2

Checks 1

Files changed 2

change-this-to-your-gh-username commented 2 minutes ago

Owner

Summary

`CatalogView` was an immutable value object expressed as a hand-written `JavaBean`-style class. It declared four fields, a constructor, getters, and boilerplate value-semantics methods.

Convert `CatalogView` to a Java record, the Java 25-era idiom for a transparent immutable data carrier. The existing `getCategories`, `getProducts`, `getSelectedCategoryId`, `getSearch`, and `hasProducts` methods remain available, and the prior `toString` format is retained, so current Java callers and Thymeleaf rendering preserve their established interface and behaviour.

This makes the value-oriented intent explicit and delegates component storage, construction, equality, and hash-code semantics to the compiler. It removes code that otherwise has to be kept in sync whenever catalog view state changes, while aligning the type with the existing record-based catalog and cart view models.

Testing

- `./mvnw test` — passed: 43 tests, 0 failures, 0 errors, 0 skipped.
- `./mvnw verify` — passed: 43 unit/integration tests and 3 Playwright E2E tests, 0 failures, 0 errors, 0 skipped.

A focused `CatalogViewTest` verifies record component access, compatibility getters, `hasProducts`, equality, hash-code semantics, and the retained string representation. Existing catalog MVC tests continue to verify unchanged server-rendered output.

Reviews

No reviews

Still in progress? Convert to draft

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Unsubscribe

You're receiving notifications because you authored this thread.

albertattard added 2 commits 14 minutes ago

Modernise catalog view as a record

Preserve catalog view string representation

All checks have passed

1 successful check

CI / verify (pull_request) Successful in 1m

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)

Still in progress? Convert to draft

change-this-to-your-gh-username commented 2 minutes ago

Owner

Summary

`CatalogView` was an immutable value object expressed as a hand-written `JavaBean`-style class. It declared four fields, a constructor, getters, and boilerplate value-semantics methods.

Convert `CatalogView` to a Java record, the Java 25-era idiom for a transparent immutable data carrier. The existing `getCategories`, `getProducts`, `getSelectedCategoryId`, `getSearch`, and `hasProducts` methods remain available, and the prior `toString` format is retained, so current Java callers and Thymeleaf rendering preserve their established interface and behaviour.

This makes the value-oriented intent explicit and delegates component storage, construction, equality, and hash-code semantics to the compiler. It removes code that otherwise has to be kept in sync whenever catalog view state changes, while aligning the type with the existing record-based catalog and cart view models.

Testing

- `./mvnw test` — passed: 43 tests, 0 failures, 0 errors, 0 skipped.
- `./mvnw verify` — passed: 43 unit/integration tests and 3 Playwright E2E tests, 0 failures, 0 errors, 0 skipped.

A focused `CatalogViewTest` verifies record component access, compatibility getters, `hasProducts`, equality, hash-code semantics, and the retained string representation. Existing catalog MVC tests continue to verify unchanged server-rendered output.

123 Copyright © 2026, Oracle and/or its affiliates | albert.attard@oracle.com

25.08.2026

Keep deterministic control outside the agent



- Put scheduling, existing-PR checks, and stop conditions in code
- Use the agent for one judgement-heavy transformation
- Bound its tools, iterations, and permissions

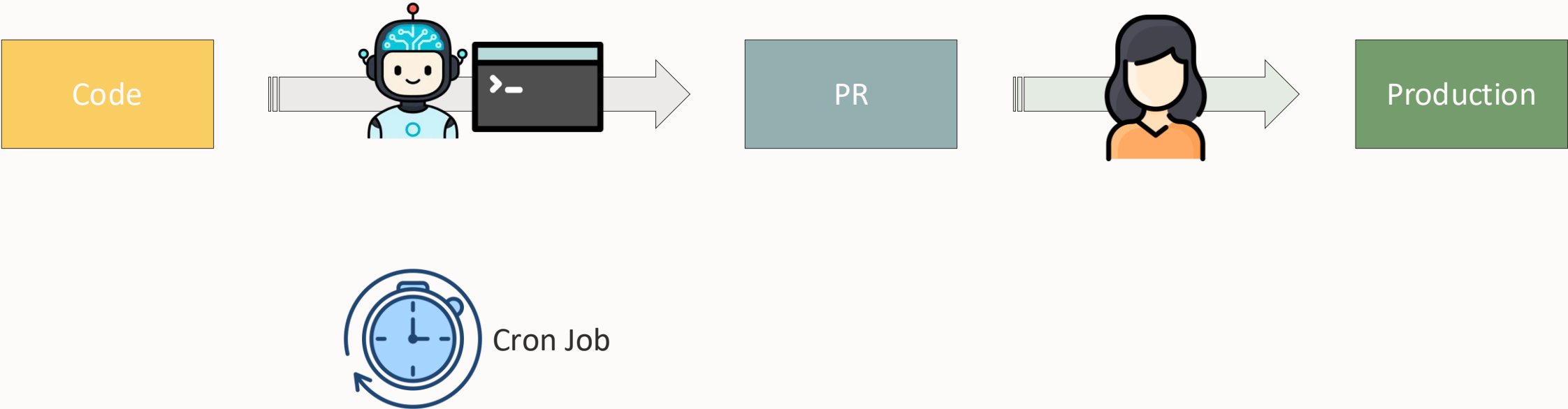
Strengthen the Test Suite

Strengthen the Test Suite



- Start only when no test-improvement PR is open
- Select one surviving mutation
- Add one test that kills it
- Verify with relevant unit and mutation tests
- Create a PR

Improve test coverage



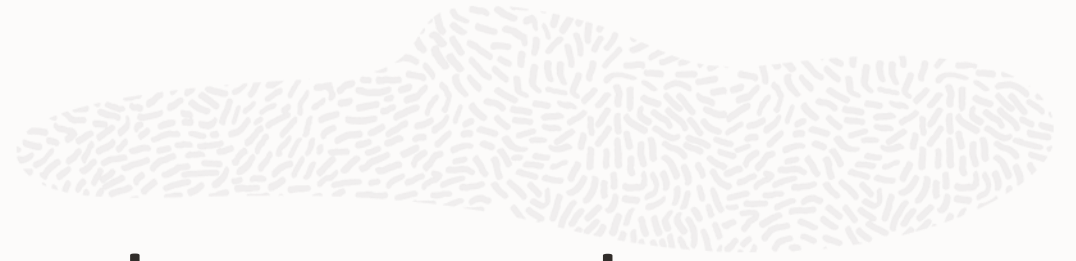
Use tools to guide and verify the agent



- Mutation testing identifies one surviving mutation
- The agent proposes one focused test
- Unit and mutation tests verify the result

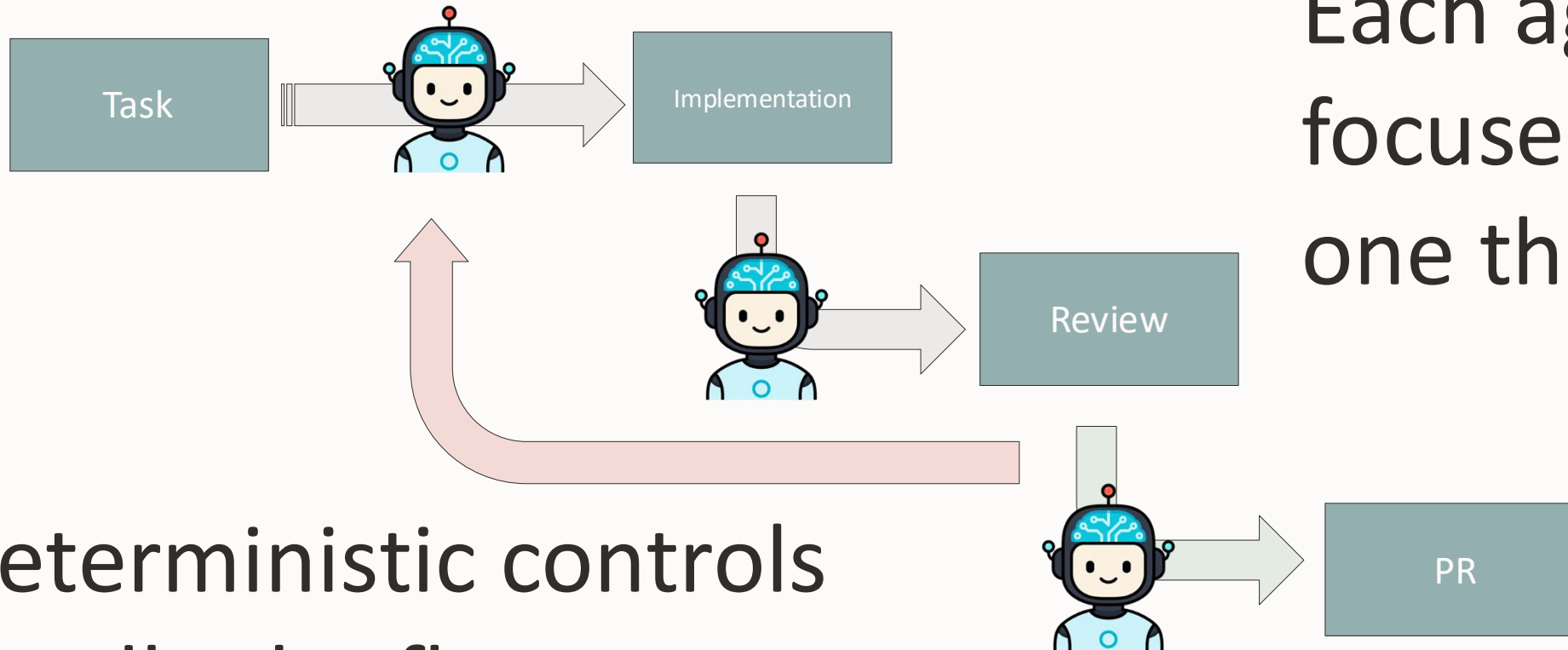
Agent Chaining

Break the workflow into bounded roles



- Implementer produces a change and evidence
- Reviewer returns approval or actionable findings
- Approval opens a PR for human review
- Findings trigger a limited revision loop

Agent Chaining



Each agent
focuses on
one thing

Deterministic controls
handle the flow

Add agents only when the hand-off helps



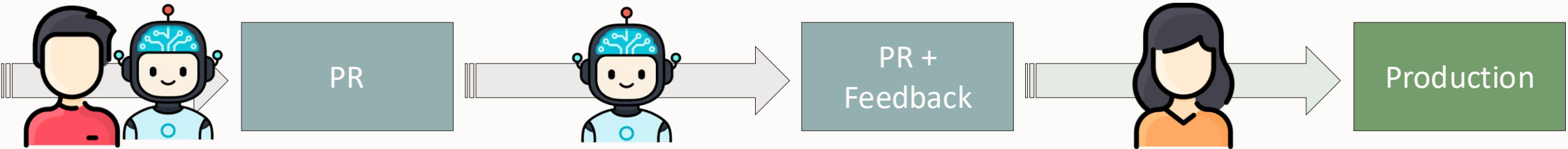
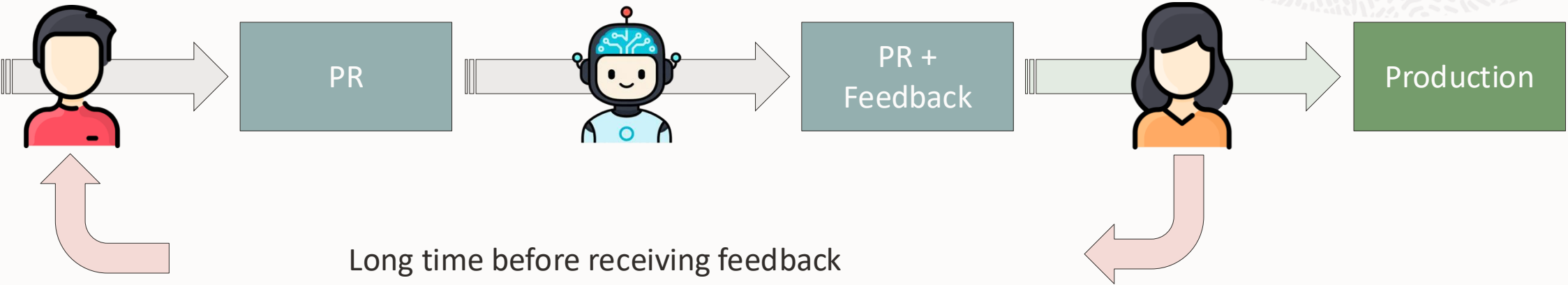
- Give each agent one bounded responsibility
- Pass explicit artefacts, not a growing shared conversation
- Use deterministic controls for retries, limits, and escalation

Code Review

Review feedback is most useful before the PR

- PR feedback arrives late
- Run review checks while coding
- Keep human PR review

Code Review



Shift Left: Make the same check available to the engineer so that they get feedback quicker!



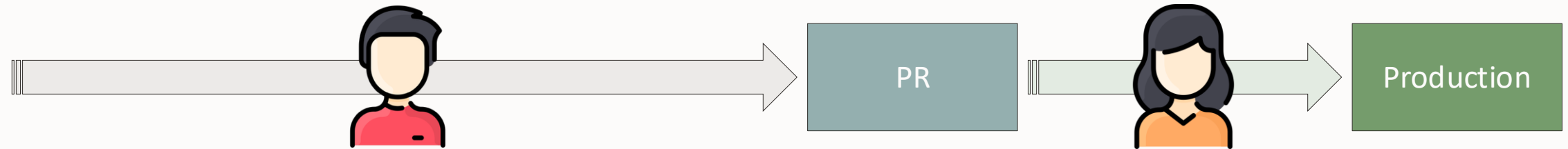
Shifting the economics of software development

Generating candidates is cheaper; choosing remains costly

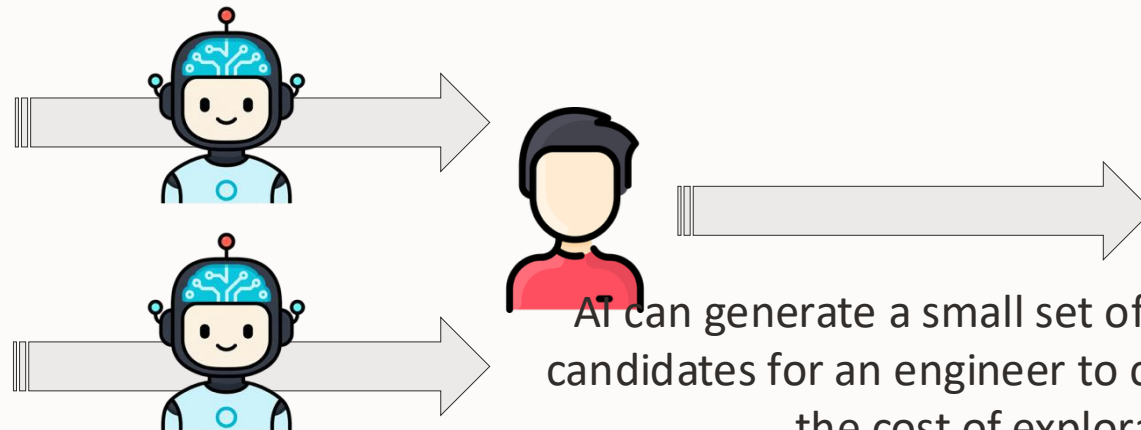


- Turning an idea into a working software change once required significant engineering time
- AI can now produce several bounded implementation candidates quickly

AI lowers the cost of exploring implementation options

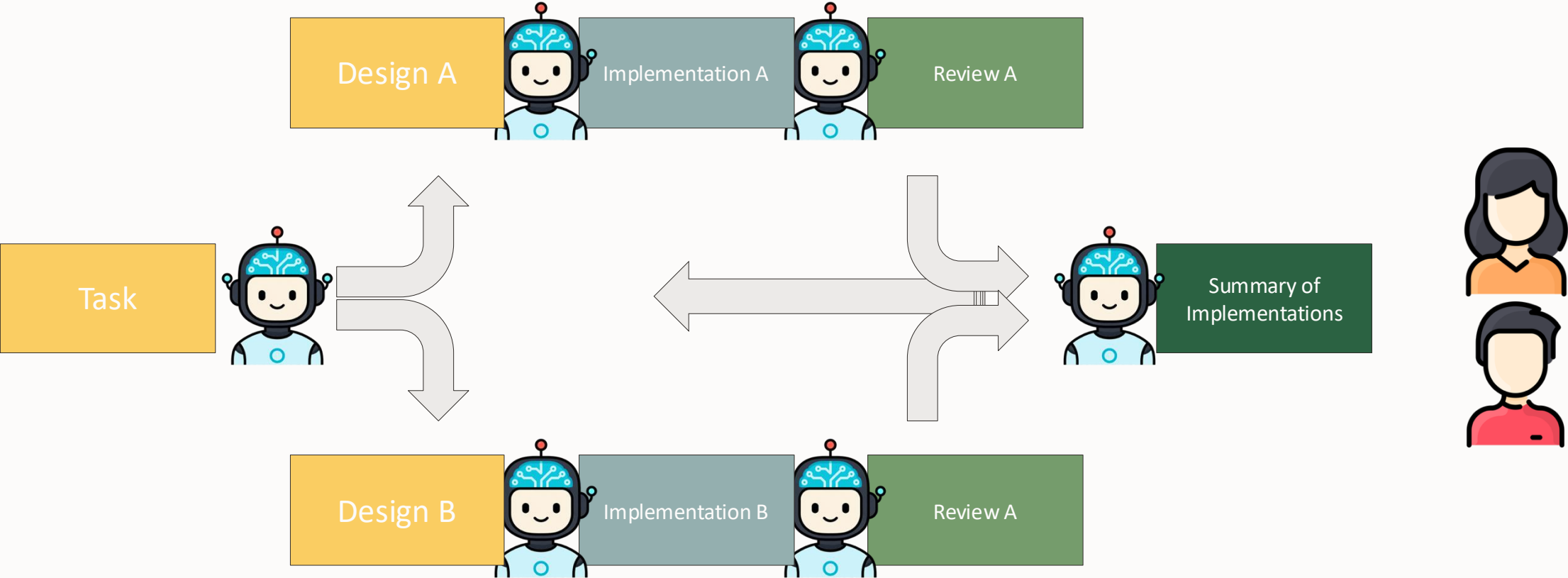


Exploring several implementation options was often expensive.



AI can generate a small set of implementation candidates for an engineer to compare, reducing the cost of exploration.

AI executes the workflow; engineers decide

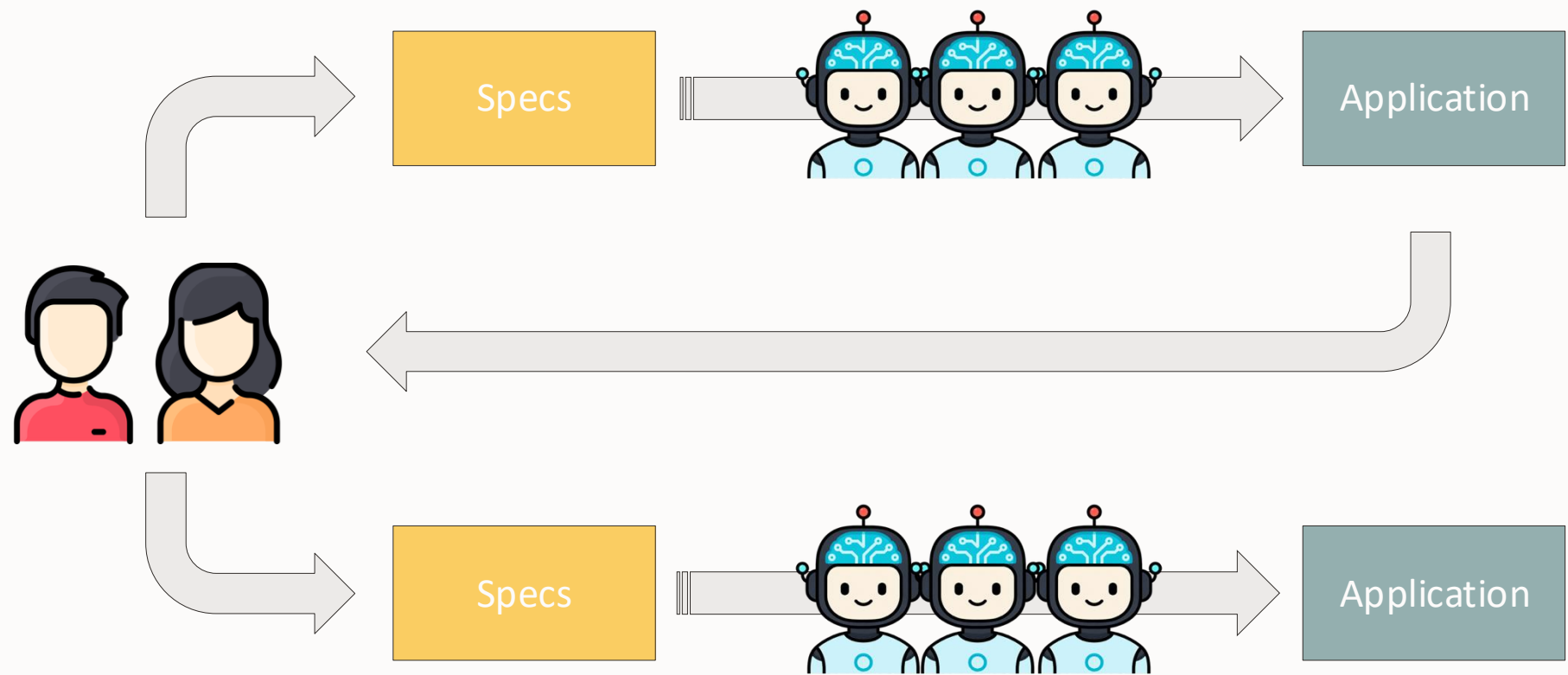


Ephemeral Applications

Engineers maintain the definition; AI regenerates the implementation

- Version behaviour, decisions, contracts, and UX
- Keep tests, guardrails, and release checks
- Regenerate implementation when the definition changes

Ephemeral Applications



Tactical delivery, strategic architecture

Tactical decisions optimise a change; strategic decisions preserve the system

Tactical	Strategic
Solves the immediate task in one area.	Protects the system's long-term coherence.
Optimises local code, tests, and delivery.	Shapes boundaries, dependencies, ownership, and quality attributes.
<i>"How can we change this component?"</i>	<i>"What must remain true across the whole system?"</i>

AI accelerates tactical work; engineers provide strategic direction

- AI optimises for the context and success criteria it can see
- Give it architectural boundaries, contracts, and constraints
- Use it to expose trade-offs; engineers make strategic decisions

Next



Proof of concept

- **Identify** an application
- **Experiment** and compare the results



How can we help

- Ad hoc conversations
- Workshops
- Knowledge Transfer
- Pairing sessions
- Hands-on



We are here to help

You are not alone
Oracle is here to help



Thank you

Albert Attard

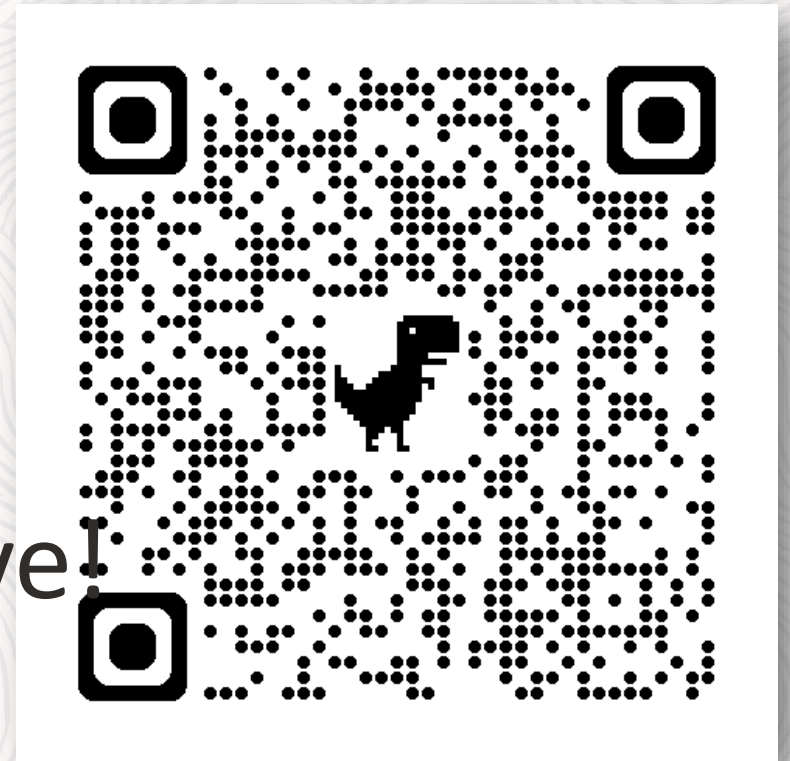
Principal Java Success Consultant

albert.attard@oracle.com



Your feedback is invaluable to us!
Please share your thoughts
through our [brief survey](#) or
by email:
albert.attard@oracle.com.

Thank you for helping us improve!



ORACLE

Our mission is to help people see
data in new ways, discover insights,
unlock endless possibilities.

